

MCP Server Onboarding Guide

Production-Grade BigQuery · GCS · Slack Access via Model Context Protocol

1. Project Architecture & Overview

What Is MCP and Why Does It Matter

The Model Context Protocol (MCP) is an open standard that separates *reasoning* from *execution*. An MCP server is a process that exposes a catalogue of typed, schema-validated **tools**. An MCP client — Claude Desktop, Cursor, Gemini CLI, or any LLM-backed product — inspects that catalogue, decides which tool to invoke and with what arguments, and sends a structured call. The server executes it and returns a structured result. The server itself contains **zero reasoning logic**: it never calls an LLM, never decides what to do next, and never chains tools together on its own. That separation is the entire value proposition for enterprise deployments — it means you can audit, rate-limit, and govern every individual API call without needing to understand or intercept the model's reasoning process.

In this project the server owns three real integrations (BigQuery, GCS, Slack) plus the safety layer that wraps them. The MCP client (Claude Desktop, Cursor, or the optional `test_client.py`) owns the conversation, the prompt, and the decision about which tool to invoke. Never mix these responsibilities.

Transport Choice: stdio over SSE/HTTP

The MCP specification supports two transports: stdio (standard input/output piped between parent and child processes) and SSE/HTTP (server-sent events over a long-lived HTTP connection). For local development and IDE integrations (Claude Desktop, Cursor) **stdio is the correct choice** for the following reasons:

- **No port management.** The host application spawns the server as a subprocess; there is no port to allocate, expose, or firewall.
- **Authentication is implicit.** The server inherits the user's shell environment, including Application Default Credentials and `.env`-sourced secrets. There is no bearer token to manage for the transport itself.
- **Process isolation.** The server lives only as long as the client needs it; the OS reclaims all resources on exit.
- **Simpler debugging.** `stderr` from the server process is visible directly in the host application's logs.

SSE/HTTP becomes appropriate when the server needs to be shared across multiple users or machines (e.g., a team deployment behind a load balancer), or when the client cannot spawn subprocesses. For an intern learning the pattern on their laptop, stdio is unambiguously correct.

SQL Safety: sqlglot over Regex

A regex-based approach to SQL safety — for example, checking that the query starts with `SELECT` and does not contain the words `DROP` or `DELETE` — fails in all of the following realistic cases:

- `SELECT * FROM t; DROP TABLE t` — the DROP follows a semicolon and a regex anchored to the start misses it entirely.
- `/* DROP TABLE t */ SELECT 1` — a comment can be used to disguise intent; regex cannot parse comment boundaries correctly.
- `WITH cte AS (DELETE FROM t RETURNING *) SELECT * FROM cte` — a CTE wraps a DML statement inside what looks like a SELECT.
- `SeLeCt` with mixed case, or `SEL/**/ECT` with an inline comment — keyword detection without a real lexer is trivially bypassed.

`sqlglot` parses the SQL string into an Abstract Syntax Tree (AST). The safety check walks the AST and verifies: (a) exactly one statement exists, and (b) the root node of that statement is a `SELECT` expression. This approach is language-agnostic across BigQuery, Snowflake, Postgres, and other dialects, and it is immune to all the obfuscation techniques above because it operates on parsed structure, not on raw text.

Library Justification

Library	Why it was chosen
<code>mcp</code> (official Python SDK)	The reference implementation; provides <code>Server</code> , <code>ListToolsRequestSchema</code> , <code>CallToolRequestSchema</code> , and the stdio runner. Using the official SDK means the server is automatically compatible with every MCP-compatible client.
<code>google-cloud-bigquery</code>	Official Google client library; supports <code>QueryJobConfig(dry_run=True)</code> natively, which is required for the cost-gating safety check before execution.
<code>google-cloud-storage</code>	Official GCS client; exposes <code>blob.size</code> as a metadata attribute populated by a lightweight <code>HEAD</code> -equivalent before any content download begins.
<code>sqlglot</code>	AST-level SQL parser; dialect-aware; actively maintained; zero transitive dependencies that conflict with the GCP stack.
<code>python-dotenv</code>	Industry-standard <code>.env</code> loader; reads <code>GCS_MAX_FILE_SIZE_BYTES</code> , <code>BQ_MAX_BYTES_SCANNED</code> , <code>SLACK_RATE_LIMIT</code> at startup without polluting the environment of the parent shell.
<code>pytest</code> + <code>pytest-mock</code>	The standard Python testing stack; <code>pytest-mock</code> wraps <code>unittest.mock</code> ergonomically and integrates cleanly with fixtures.

2. Repository & Folder Structure

ASCII Tree

```
mcp_server/
├── server.py
├── tools/
│   ├── __init__.py
│   ├── query_bigquery.py
│   └── list_gcs_objects.py
```

```
|   ├── read_gcs_object.py
|   ├── send_slack_message.py
|   └── safety/
|       ├── __init__.py
|       ├── sql_safety.py
|       ├── gcs_safety.py
|       └── slack_safety.py
|   └── config/
|       ├── __init__.py
|       └── settings.py
|   └── utils/
|       ├── __init__.py
|       ├── logger.py
|       └── error_format.py
|   └── tests/
|       ├── __init__.py
|       ├── test_query_bigquery.py
|       ├── test_list_gcs_objects.py
|       ├── test_read_gcs_object.py
|       ├── test_send_slack_message.py
|       └── test_safety.py
|   └── test_client.py      ← optional / stretch goal (Groq-based)
|   └── README.md
|   └── error_format.md
|   └── questions.md
|   └── requirements.txt
|   └── .env.example
|   └── .gitignore
|   └── setup.sh
```

setup.sh

Save the following as `setup.sh` in the project root, then run `chmod +x setup.sh && ./setup.sh`.

```
#!/usr/bin/env bash
# setup.sh – scaffolds the entire mcp_server project structure
# Run from the directory where you want the project to live.
# Usage: chmod +x setup.sh && ./setup.sh

set -euo pipefail

PROJECT="mcp_server"

echo "==> Creating project root: $PROJECT"
mkdir -p "$PROJECT"

echo "==> Creating package directories"
mkdir -p "$PROJECT/tools"
mkdir -p "$PROJECT/safety"
mkdir -p "$PROJECT/config"
mkdir -p "$PROJECT/utils"
```

```
mkdir -p "$PROJECT/tests"

echo "==> Touching Python source files"
touch "$PROJECT/server.py"

touch "$PROJECT/tools/__init__.py"
touch "$PROJECT/tools/query_bigquery.py"
touch "$PROJECT/tools/list_gcs_objects.py"
touch "$PROJECT/tools/read_gcs_object.py"
touch "$PROJECT/tools/send_slack_message.py"

touch "$PROJECT/safety/__init__.py"
touch "$PROJECT/safety/sql_safety.py"
touch "$PROJECT/safety/gcs_safety.py"
touch "$PROJECT/safety/slack_safety.py"

touch "$PROJECT/config/__init__.py"
touch "$PROJECT/config/settings.py"

touch "$PROJECT/utils/__init__.py"
touch "$PROJECT/utils/logger.py"
touch "$PROJECT/utils/error_format.py"

touch "$PROJECT/tests/__init__.py"
touch "$PROJECT/tests/test_query_bigquery.py"
touch "$PROJECT/tests/test_list_gcs_objects.py"
touch "$PROJECT/tests/test_read_gcs_object.py"
touch "$PROJECT/tests/test_send_slack_message.py"
touch "$PROJECT/tests/test_safety.py"

echo "==> Touching project-level files"
touch "$PROJECT/test_client.py"
touch "$PROJECT/README.md"
touch "$PROJECT/error_format.md"
touch "$PROJECT/questions.md"
touch "$PROJECT/requirements.txt"
touch "$PROJECT/.env.example"
touch "$PROJECT/.gitignore"

echo "==> Creating Python virtual environment (myenv) inside $PROJECT"
cd "$PROJECT"
python3 -m venv myenv

echo ""
echo "✅ Scaffold complete."
echo ""
echo "Next steps:"
echo "  cd $PROJECT"
echo "  source myenv/bin/activate"
echo "  pip install -r requirements.txt"
echo "  # Fill in .env from .env.example"
echo "  python server.py"
```

3. Production-Ready Implementation Code

requirements.txt

```
mcp>=1.0.0
google-cloud-bigquery>=3.11.0
google-cloud-storage>=2.10.0
sqlglot>=23.0.0
python-dotenv>=1.0.0
pytest>=7.4.0
pytest-mock>=3.11.0
groq>=0.4.0
```

.env.example

```
# BigQuery: reject queries that would scan more than this many bytes
(default 100 MB)
BQ_MAX_BYTES_SCANNED=104857600

# GCS: reject file reads larger than this many bytes (default 10 MB)
GCS_MAX_FILE_SIZE_BYTES=10485760

# Slack rate limit: max messages per window per channel
SLACK_RATE_LIMIT_MAX_CALLS=5
SLACK_RATE_LIMIT_PERIOD_SECONDS=60

# Google auth: set to your service account key path for non-interactive
envs
# Leave unset when using: gcloud auth application-default login
# GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json

# Optional: only needed if you run test_client.py
# GROQ_API_KEY=gsk_...
```

.gitignore

```
myenv/  
__pycache__/  
*.pyc  
*.pyo  
.pytest_cache/  
.env  
output.log  
*.egg-info/  
dist/  
build/  
.DS_Store
```

utils/logger.py

```
"""  
utils/logger.py  
  
Configures a single application-wide logger that writes structured log  
lines  
simultaneously to stdout (StreamHandler) and output.log (FileHandler).  
  
Format: %(asctime)s | %(levelname)s | %(message)s  
  
Import and use:  
    from utils.logger import get_logger  
    logger = get_logger(__name__)  
    logger.info("Server started")  
"""  
  
import logging  
import sys  
from pathlib import Path  
  
_LOG_FILE = Path(__file__).parent.parent / "output.log"  
_FORMAT = "%(asctime)s | %(levelname)s | %(message)s"  
_DATE_FORMAT = "%Y-%m-%d %H:%M:%S"  
  
# Module-level flag so handlers are only attached once regardless of how  
many  
# times get_logger() is called across imports.  
_configured = False  
  
def _configure_root() -> None:  
    global _configured  
    if _configured:  
        return  
  
    root = logging.getLogger("mcp_server")
```

```

root.setLevel(logging.DEBUG)

formatter = logging.Formatter(fmt=_FORMAT, datefmt=_DATE_FORMAT)

# Terminal handler – INFO and above, so DEBUG messages don't flood the
console
stream_handler = logging.StreamHandler(sys.stdout)
stream_handler.setLevel(logging.INFO)
stream_handler.setFormatter(formatter)

# File handler – DEBUG and above, so every detail is persisted to
output.log
file_handler = logging.FileHandler(_LOG_FILE, mode="a", encoding="utf-
8")
file_handler.setLevel(logging.DEBUG)
file_handler.setFormatter(formatter)

root.addHandler(stream_handler)
root.addHandler(file_handler)

# Suppress noisy third-party loggers at WARNING level
for noisy in ("google.auth", "google.api_core", "urllib3", "httpx"):
    logging.getLogger(noisy).setLevel(logging.WARNING)

_configured = True

def get_logger(name: str) -> logging.Logger:
    """
    Returns a child logger under the 'mcp_server' namespace.
    All child loggers propagate to the root logger which holds the
    handlers.
    """
    _configure_root()
    return logging.getLogger(f"mcp_server.{name}")

```

utils/error_format.py

```

"""
utils/error_format.py

Defines the ONE canonical error envelope used by every tool in this server.
Import make_error() and make_success() everywhere; never construct dicts
inline.

Error envelope shape:
{
    "success": False,
    "error_code": str,          # VALIDATION_ERROR | SAFETY_LIMIT_EXCEEDED |
                                DOWNSTREAM_ERROR

```

```

    "message": str,          # human-readable explanation
    "tool": str,             # which tool raised this error
    "details": dict          # optional extra context (bytes processed,
limit, etc.)
}

```

Success envelope shape:

```

{
    "success": True,
    "tool": str,
    "data": any,             # tool-specific payload
    "meta": dict             # optional metadata (job_id,
execution_time_ms, etc.)
}
"""

```

```

from typing import Any

```

```

# Canonical error code constants – import these instead of using raw
strings

```

```

VALIDATION_ERROR = "VALIDATION_ERROR"
SAFETY_LIMIT_EXCEEDED = "SAFETY_LIMIT_EXCEEDED"
DOWNSTREAM_ERROR = "DOWNSTREAM_ERROR"

```

```

def make_error(
    tool: str,
    error_code: str,
    message: str,
    details: dict | None = None,
) -> dict:
    """
    Build a structured error envelope.

    Args:
        tool:          Name of the tool that produced this error (e.g.
"query_bigquery").
        error_code:    One of VALIDATION_ERROR, SAFETY_LIMIT_EXCEEDED,
DOWNSTREAM_ERROR.
        message:       A clear, human-readable explanation of what went
wrong.
        details:       Optional dict with extra context (bytes_processed,
limit, etc.).

    Returns:
        A dict conforming to the canonical error envelope schema.
    """
    return {
        "success": False,
        "error_code": error_code,
        "message": message,
        "tool": tool,
        "details": details or {},
    }

```



```

    }

def make_success(
    tool: str,
    data: Any,
    meta: dict | None = None,
) -> dict:
    """
    Build a structured success envelope.

    Args:
        tool: Name of the tool that produced this result.
        data: The tool-specific payload (rows, object list, file content,
        etc.).
        meta: Optional metadata (job_id, execution_time_ms, row_count,
        etc.).

    Returns:
        A dict conforming to the canonical success envelope schema.
    """
    return {
        "success": True,
        "tool": tool,
        "data": data,
        "meta": meta or {},
    }

```

config/settings.py

```

"""
config/settings.py

Loads all runtime configuration from environment variables (populated via
.env
at startup). Every module that needs a limit or credential imports from
here;
nothing reads os.environ directly outside this file.
"""

import os
from pathlib import Path

from dotenv import load_dotenv

# Load .env from the project root (two levels above this file: config/ ->
mcp_server/)
_PROJECT_ROOT = Path(__file__).parent.parent
load_dotenv(_PROJECT_ROOT / ".env")

```

```
def _get_int(key: str, default: int) -> int:
    raw = os.environ.get(key, "")
    if not raw.strip():
        return default
    try:
        return int(raw)
    except ValueError:
        raise ValueError(
            f"[settings] Environment variable {key!r} must be an integer;
got {raw!r}"
        )

# — BigQuery


---


# Maximum bytes a query may scan. Queries whose dry-run estimate exceeds
this
# are rejected before execution. Default: 100 MB.
BQ_MAX_BYTES_SCANNED: int = _get_int("BQ_MAX_BYTES_SCANNED", 104_857_600)

# BigQuery project ID – inferred from Application Default Credentials when
unset.
BQ_PROJECT_ID: str | None = os.environ.get("BQ_PROJECT_ID") or None

# — Google Cloud Storage


---


# Maximum file size (bytes) allowed for read_gcs_object. Default: 10 MB.
GCS_MAX_FILE_SIZE_BYTES: int = _get_int("GCS_MAX_FILE_SIZE_BYTES",
10_485_760)

# — Slack rate limiting


---


# Token bucket: max N calls per M seconds, per channel.
SLACK_RATE_LIMIT_MAX_CALLS: int = _get_int("SLACK_RATE_LIMIT_MAX_CALLS", 5)
SLACK_RATE_LIMIT_PERIOD_SECONDS: int =
_get_int("SLACK_RATE_LIMIT_PERIOD_SECONDS", 60)

# — Optional


---


# GOOGLE_APPLICATION_CREDENTIALS is read by google-auth automatically; we
just
# surface it here for documentation purposes.
GOOGLE_APPLICATION_CREDENTIALS: str | None = os.environ.get(
    "GOOGLE_APPLICATION_CREDENTIALS"
) or None

# Groq API key – only needed for the optional test_client.py
GROQ_API_KEY: str | None = os.environ.get("GROQ_API_KEY") or None
```

```
# config/__init__.py – marks config as a package; re-exports settings for
convenience
from config.settings import (
    BQ_MAX_BYTES_SCANNED,
    BQ_PROJECT_ID,
    GCS_MAX_FILE_SIZE_BYTES,
    SLACK_RATE_LIMIT_MAX_CALLS,
    SLACK_RATE_LIMIT_PERIOD_SECONDS,
    GROQ_API_KEY,
)

__all__ = [
    "BQ_MAX_BYTES_SCANNED",
    "BQ_PROJECT_ID",
    "GCS_MAX_FILE_SIZE_BYTES",
    "SLACK_RATE_LIMIT_MAX_CALLS",
    "SLACK_RATE_LIMIT_PERIOD_SECONDS",
    "GROQ_API_KEY",
]
```

utils/__init__.py

```
# utils/__init__.py – package marker
```

tools/__init__.py

```
# tools/__init__.py – package marker
```

safety/__init__.py

```
# safety/__init__.py – package marker
```

safety/sql_safety.py

```
"""
safety/sql_safety.py

AST-level SQL safety validation using sqlglot.
```

Rules enforced:

1. The input must parse successfully as valid SQL.
2. Exactly one statement must be present (no multi-statement batches).
3. The single statement must be a SELECT (not INSERT/UPDATE/DELETE/MERGE/CREATE/DROP/ALTER/TRUNCATE or any DDL/DML variant).

Why sqlglot instead of regex:

- Handles comments, mixed case, CTEs, and dialect variations correctly.
- Operates on the parsed AST, making it impossible to bypass with formatting tricks.
- dialect="bigquery" enables BigQuery-specific parsing (STRUCT, ARRAY, backticks, etc.).

"""

```
import sqlglot
import sqlglot.expressions as exp
```

```
from utils.error_format import VALIDATION_ERROR, make_error
```

```
_TOOL = "query_bigquery"
```

```
def validate_select_only(sql: str) -> dict | None:
```

"""

Validates that `sql` is a single, read-only SELECT statement.

Returns:

None if the SQL is safe.

A structured error envelope (dict) if validation fails – the caller must return this envelope immediately without proceeding to

execution.

"""

```
if not sql or not sql.strip():
    return make_error(
        tool=_TOOL,
        error_code=VALIDATION_ERROR,
        message="SQL query must not be empty.",
        details={"sql_preview": ""},
    )
```

```
try:
```

```
    statements = sqlglot.parse(sql, dialect="bigquery",
error_level=sqlglot.ErrorLevel.RAISE)
```

```
except sqlglot.errors.ParseError as exc:
```

```
    return make_error(
        tool=_TOOL,
        error_code=VALIDATION_ERROR,
        message=f"SQL failed to parse: {exc}",
        details={"sql_preview": sql[:200]},
    )
```

```
# Rule 1: Exactly one statement
```

```
if len(statements) != 1:
```

```
    return make_error(
```

```

        tool=_T00L,
        error_code=VALIDATION_ERROR,
        message=(
            f"Exactly one SQL statement is allowed; received
{len(statements)}. "
            "Multi-statement batches are blocked."
        ),
        details={"statement_count": len(statements), "sql_preview":
sql[:200]},
    )

    statement = statements[0]

    # Rule 2: Statement must be a SELECT (includes CTEs that resolve to
    # SELECT)
    # sqlglot represents a bare SELECT as exp.Select and a CTE+SELECT as
    # exp.Select
    # with a With clause parent. We check the outermost expression type.
    if not isinstance(statement, exp.Select):
        actual_type = type(statement).__name__
        return make_error(
            tool=_T00L,
            error_code=VALIDATION_ERROR,
            message=(
                f"Only SELECT statements are permitted. "
                f"Detected statement type: {actual_type}. "
                "INSERT, UPDATE, DELETE, MERGE, CREATE, DROP, ALTER, and
TRUNCATE are blocked."
            ),
            details={"detected_type": actual_type, "sql_preview":
sql[:200]},
        )

    # Rule 3: Even inside a SELECT, scan for any DML sub-expressions
    # This catches: WITH cte AS (DELETE FROM t RETURNING *) SELECT * FROM
    cte
    forbidden_types = (
        exp.Insert,
        exp.Update,
        exp.Delete,
        exp.Merge,
        exp.Create,
        exp.Drop,
        exp.AlterTable,
        exp.TruncateTable,
    )
    for node in statement.walk():
        if isinstance(node, forbidden_types):
            node_type = type(node).__name__
            return make_error(
                tool=_T00L,
                error_code=VALIDATION_ERROR,
                message=(
                    f"Forbidden DML expression {node_type!r} detected

```

```

inside the query. "
            "All data-modifying operations are blocked."
        ),
        details={"forbidden_node": node_type, "sql_preview":
sql[:200]}},
    )

    return None # All checks passed

```

safety/gcs_safety.py

```

"""
safety/gcs_safety.py

Pre-download size check for GCS objects.

The blob's `size` attribute is populated when the Blob object is fetched
via
`bucket.get_blob()` (which issues a lightweight metadata request, not a
content
download). We check this BEFORE calling `blob.download_as_bytes()`,
ensuring
that large files are rejected without consuming memory or egress bandwidth.
"""

from config.settings import GCS_MAX_FILE_SIZE_BYTES
from utils.error_format import SAFETY_LIMIT_EXCEEDED, make_error

_T00L = "read_gcs_object"

def check_blob_size(blob_size_bytes: int, path: str) -> dict | None:
    """
    Checks whether a GCS object's size is within the configured limit.

    Args:
        blob_size_bytes: The size reported in the blob's metadata (bytes).
        path: The object path, included in the error for
context.

    Returns:
        None if the size is acceptable.
        A structured error envelope (dict) if the file is too large.
    """
    if blob_size_bytes > GCS_MAX_FILE_SIZE_BYTES:
        return make_error(
            tool=_T00L,
            error_code=SAFETY_LIMIT_EXCEEDED,
            message=(
                f"File at {path!r} is {blob_size_bytes:,} bytes, which

```

```

exceeds the "
        f"configured maximum of {GCS_MAX_FILE_SIZE_BYTES:,} bytes "
        f"({GCS_MAX_FILE_SIZE_BYTES / 1_048_576:.1f} MB). "
        "Increase GCS_MAX_FILE_SIZE_BYTES in .env if you need to
read larger files."
    ),
    details={
        "file_size_bytes": blob_size_bytes,
        "limit_bytes": GCS_MAX_FILE_SIZE_BYTES,
        "path": path,
    },
)
return None

```

safety/slack_safety.py

```

"""
safety/slack_safety.py

Token-bucket rate limiter for the Slack stub tool.

Why token bucket instead of fixed window:
- A fixed window allows a burst of N calls at the end of one window and
  N calls at the start of the next, effectively allowing 2N calls in a
  short period (the "boundary burst" problem).
- A token bucket smooths this: tokens refill continuously at rate
  (max_calls / period_seconds) per second, and each call consumes one
  token.
  If a burst depletes the bucket, subsequent calls must wait for refill.

The bucket state is held in memory (a dict keyed by channel). This is
appropriate for a single-process MCP server. For a multi-process deployment
you would move the state to Redis or a similar shared store.

Configuration (from .env / settings.py):
    SLACK_RATE_LIMIT_MAX_CALLS          default 5
    SLACK_RATE_LIMIT_PERIOD_SECONDS     default 60
"""

import time
from threading import Lock
from typing import NamedTuple

from config.settings import SLACK_RATE_LIMIT_MAX_CALLS,
SLACK_RATE_LIMIT_PERIOD_SECONDS
from utils.error_format import SAFETY_LIMIT_EXCEEDED, make_error

_TOOL = "send_slack_message"

```

```

class _BucketState(NamedTuple):
    tokens: float          # current number of available tokens (can be
fractional)
    last_refill: float     # Unix timestamp of the last refill calculation

class TokenBucketRateLimiter:
    """
    Per-channel token bucket rate limiter.

    A channel starts with `max_calls` tokens. Each message call consumes
one
token. Tokens refill at `max_calls / period_seconds` per second, capped
at
`max_calls`. Calls that would reduce tokens below 0 are rejected.
    """

    def __init__(self, max_calls: int, period_seconds: int) -> None:
        self._max_calls = max_calls
        self._rate = max_calls / period_seconds # tokens per second
        self._buckets: dict[str, _BucketState] = {}
        self._lock = Lock()

    def _refill(self, state: _BucketState, now: float) -> float:
        """Compute new token count after refilling since last_refill."""
        elapsed = now - state.last_refill
        new_tokens = state.tokens + elapsed * self._rate
        return min(new_tokens, self._max_calls)

    def check_and_consume(self, channel: str) -> dict | None:
        """
        Attempts to consume one token for `channel`.

        Returns:
            None if the call is allowed (token consumed).
            A structured error envelope if the rate limit is exceeded.
        """
        with self._lock:
            now = time.monotonic()
            if channel not in self._buckets:
                self._buckets[channel] = _BucketState(
                    tokens=float(self._max_calls), last_refill=now
                )

            state = self._buckets[channel]
            current_tokens = self._refill(state, now)

            if current_tokens < 1.0:
                # Calculate seconds until next token is available
                tokens_needed = 1.0 - current_tokens
                seconds_until_available = tokens_needed / self._rate
                return make_error(
                    tool=_T00L,
                    error_code=SAFETY_LIMIT_EXCEEDED,

```



```

        message=(
            f"Rate limit exceeded for channel {channel!r}. "
            f"Maximum {self._max_calls} messages per "
            f"{int(self._rate ** -1 * self._max_calls)}
seconds. "
            f"Retry in approximately
{seconds_until_available:.1f} seconds."
        ),
        details={
            "channel": channel,
            "tokens_remaining": round(current_tokens, 3),
            "max_calls": self._max_calls,
            "retry_after_seconds":
round(seconds_until_available, 2),
        },
    )

    # Consume one token
    self._buckets[channel] = _BucketState(
        tokens=current_tokens - 1.0, last_refill=now
    )
    return None # Call is allowed

def reset(self, channel: str | None = None) -> None:
    """Resets bucket state. Useful in tests to start with a clean
    slate."""
    with self._lock:
        if channel is None:
            self._buckets.clear()
        else:
            self._buckets.pop(channel, None)

# Module-level singleton – server.py imports this instance
_rate_limiter = TokenBucketRateLimiter(
    max_calls=SLACK_RATE_LIMIT_MAX_CALLS,
    period_seconds=SLACK_RATE_LIMIT_PERIOD_SECONDS,
)

def check_slack_rate_limit(channel: str) -> dict | None:
    """
    Public entry point for rate-limit checks.

    Returns:
        None if the call is allowed.
        A structured error envelope if the limit is exceeded.
    """
    return _rate_limiter.check_and_consume(channel)

def reset_rate_limiter(channel: str | None = None) -> None:
    """
    Resets the in-memory rate limiter. Exposed for test isolation.

```

```

    Args:
        channel: If provided, resets only that channel. If None, resets all
channels.
    """
    _rate_limiter.reset(channel)

```

tools/query_bigquery.py

```

"""
tools/query_bigquery.py

MCP tool: query_bigquery

Executes a read-only BigQuery SELECT query with two mandatory safety gates:

1. SQL validation (sqlglot AST): rejects anything that is not a single
SELECT.
2. Dry-run cost check: rejects queries whose estimated scan exceeds
BQ_MAX_BYTES_SCANNED.

Only if both gates pass does the actual query execute.

Returned success payload:
{
    "success": true,
    "tool": "query_bigquery",
    "data": {
        "rows": [ { "col": value, ... }, ... ],
        "row_count": int,
        "schema": [ { "name": str, "field_type": str, "mode": str }, ... ]
    },
    "meta": {
        "job_id": str,
        "bytes_processed": int,
        "execution_time_ms": float,
        "project": str
    }
}
"""

import time
from datetime import datetime, date, timezone
from decimal import Decimal

from google.cloud import bigquery
from google.cloud.exceptions import GoogleCloudError

from config.settings import BQ_MAX_BYTES_SCANNED, BQ_PROJECT_ID
from safety.sql_safety import validate_select_only
from utils.error_format import DOWNSTREAM_ERROR, SAFETY_LIMIT_EXCEEDED,

```

```

make_error, make_success
from utils.logger import get_logger

logger = get_logger("tools.query_bigquery")

TOOL_NAME = "query_bigquery"

# JSON schema for MCP tool input validation
INPUT_SCHEMA = {
    "type": "object",
    "properties": {
        "sql": {
            "type": "string",
            "description": (
                "A single, read-only BigQuery SELECT statement. "
                "INSERT, UPDATE, DELETE, MERGE, DDL, and multi-statement "
                "batches are rejected."
            ),
            "minLength": 1,
        },
    },
    "required": ["sql"],
    "additionalProperties": False,
}

def _serialize_row(row: bigquery.Row) -> dict:
    """
    Converts a BigQuery Row to a JSON-serialisable dict.
    BigQuery can return datetime, date, Decimal, and bytes values which are
    not natively JSON-serialisable; we convert them here.
    """
    result = {}
    for key, value in row.items():
        if isinstance(value, (datetime, date)):
            result[key] = value.isoformat()
        elif isinstance(value, Decimal):
            result[key] = float(value)
        elif isinstance(value, bytes):
            result[key] = value.hex()
        else:
            result[key] = value
    return result

def run(sql: str) -> dict:
    """
    Executes a BigQuery SELECT query after validating SQL safety and cost.

    Args:
        sql: The SQL query string to execute.

    Returns:
        A structured success or error envelope dict.

```

```

"""
logger.info("query_bigquery invoked | sql_preview=%r", sql[:120])

# — Gate 1: SQL validation (fail-fast)


---


validation_error = validate_select_only(sql)
if validation_error is not None:
    logger.warning(
        "query_bigquery rejected by SQL safety | error_code=%s |
message=%s",
        validation_error["error_code"],
        validation_error["message"],
    )
    return validation_error

# — Gate 2: BigQuery dry-run cost check (fail-fast)


---


try:
    client = bigquery.Client(project=BQ_PROJECT_ID)
    dry_run_config = bigquery.QueryJobConfig(
        dry_run=True,
        use_query_cache=False,
    )
    dry_run_job = client.query(sql, job_config=dry_run_config)
    estimated_bytes = dry_run_job.total_bytes_processed
except GoogleCloudError as exc:
    logger.error("query_bigquery dry-run failed | error=%s", exc)
    return make_error(
        tool=TOOL_NAME,
        error_code=DOWNSTREAM_ERROR,
        message=f"BigQuery dry-run request failed: {exc}",
        details={"exception_type": type(exc).__name__},
    )
except Exception as exc:
    logger.error("query_bigquery dry-run unexpected error | error=%s",
exc)
    return make_error(
        tool=TOOL_NAME,
        error_code=DOWNSTREAM_ERROR,
        message=f"Unexpected error during BigQuery dry-run: {exc}",
        details={"exception_type": type(exc).__name__},
    )

logger.debug(
    "query_bigquery dry-run complete | estimated_bytes=%d | limit=%d",
    estimated_bytes,
    BQ_MAX_BYTES_SCANNED,
)

if estimated_bytes > BQ_MAX_BYTES_SCANNED:
    logger.warning(
        "query_bigquery rejected by cost gate | estimated_bytes=%d |
limit=%d",
        estimated_bytes,

```

```

        BQ_MAX_BYTES_SCANNED,
    )
    return make_error(
        tool=T00L_NAME,
        error_code=SAFETY_LIMIT_EXCEEDED,
        message=(
            f"Query would scan {estimated_bytes:,} bytes "
            f"({estimated_bytes / 1_048_576:.1f} MB), exceeding the "
            f"configured limit of {BQ_MAX_BYTES_SCANNED:,} bytes "
            f"({BQ_MAX_BYTES_SCANNED / 1_048_576:.1f} MB). "
            "Refine the query with a WHERE clause or partition filter "
            "to reduce scan volume."
        ),
        details={
            "estimated_bytes": estimated_bytes,
            "limit_bytes": BQ_MAX_BYTES_SCANNED,
            "estimated_mb": round(estimated_bytes / 1_048_576, 2),
            "limit_mb": round(BQ_MAX_BYTES_SCANNED / 1_048_576, 2),
        },
    )

```

— Execution

```

try:
    exec_start = time.perf_counter()
    job = client.query(sql)
    rows_raw = list(job.result())
    exec_elapsed_ms = (time.perf_counter() - exec_start) * 1000

    rows = [_serialize_row(r) for r in rows_raw]
    schema = [
        {
            "name": field.name,
            "field_type": field.field_type,
            "mode": field.mode,
        }
        for field in job.schema
    ] if job.schema else []

    logger.info(
        "query_bigquery success | job_id=%s | rows=%d | "
        "bytes_processed=%d | elapsed_ms=%.1f",
        job.job_id,
        len(rows),
        job.total_bytes_processed or 0,
        exec_elapsed_ms,
    )

    return make_success(
        tool=T00L_NAME,
        data={
            "rows": rows,
            "row_count": len(rows),
            "schema": schema,

```

```

        },
        meta={
            "job_id": job.job_id,
            "bytes_processed": job.total_bytes_processed or 0,
            "execution_time_ms": round(exec_elapsed_ms, 2),
            "project": client.project,
        },
    )

except GoogleCloudError as exc:
    logger.error("query_bigquery execution failed | error=%s", exc)
    return make_error(
        tool=TOOL_NAME,
        error_code=DOWNSTREAM_ERROR,
        message=f"BigQuery query execution failed: {exc}",
        details={"exception_type": type(exc).__name__},
    )
except Exception as exc:
    logger.error("query_bigquery unexpected execution error | error=%s", exc)
    return make_error(
        tool=TOOL_NAME,
        error_code=DOWNSTREAM_ERROR,
        message=f"Unexpected error during query execution: {exc}",
        details={"exception_type": type(exc).__name__},
    )

```

tools/list_gcs_objects.py

```

"""
tools/list_gcs_objects.py

MCP tool: list_gcs_objects

Lists GCS objects under a given bucket/prefix.
Returns name, size, content_type, and updated timestamp for each object.

Returned success payload:
{
    "success": true,
    "tool": "list_gcs_objects",
    "data": {
        "objects": [
            {
                "name": str,
                "size_bytes": int,
                "content_type": str,
                "updated": str (ISO-8601)
            },
            ...

```

```

        ],
        "object_count": int
    },
    "meta": {
        "bucket": str,
        "prefix": str
    }
}
"""

from google.cloud import storage
from google.cloud.exceptions import GoogleCloudError, NotFound

from utils.error_format import DOWNSTREAM_ERROR, VALIDATION_ERROR,
make_error, make_success
from utils.logger import get_logger

logger = get_logger("tools.list_gcs_objects")

TOOL_NAME = "list_gcs_objects"

INPUT_SCHEMA = {
    "type": "object",
    "properties": {
        "bucket": {
            "type": "string",
            "description": "GCS bucket name (without gs:// prefix).",
            "minLength": 1,
        },
        "prefix": {
            "type": "string",
            "description": (
                "Object name prefix to filter results (e.g. 'data/2024/')."
                "Use an empty string to list all objects in the bucket."
            ),
        },
    },
    "required": ["bucket", "prefix"],
    "additionalProperties": False,
}

def run(bucket: str, prefix: str) -> dict:
    """
    Lists GCS objects in `bucket` under `prefix`.

    Args:
        bucket: GCS bucket name.
        prefix: Object name prefix (may be empty string for no filter).

    Returns:
        A structured success or error envelope dict.
    """

```

```

    logger.info("list_gcs_objects invoked | bucket=%r | prefix=%r", bucket,
prefix)

# — Input validation (fail-fast)


---


if not bucket or not bucket.strip():
    return make_error(
        tool=TOOL_NAME,
        error_code=VALIDATION_ERROR,
        message="'bucket' must be a non-empty string.",
        details={},
    )

# — Execution (graceful degradation)


---


try:
    client = storage.Client()
    blobs = list(client.list_blobs(bucket, prefix=prefix or None))

    objects = []
    for blob in blobs:
        updated_iso = blob.updated.isoformat() if blob.updated else
None
        objects.append(
            {
                "name": blob.name,
                "size_bytes": blob.size,
                "content_type": blob.content_type,
                "updated": updated_iso,
            }
        )

    logger.info(
        "list_gcs_objects success | bucket=%r | prefix=%r |
object_count=%d",
        bucket,
        prefix,
        len(objects),
    )

    return make_success(
        tool=TOOL_NAME,
        data={"objects": objects, "object_count": len(objects)},
        meta={"bucket": bucket, "prefix": prefix},
    )

except NotFound:
    logger.error("list_gcs_objects bucket not found | bucket=%r",
bucket)
    return make_error(
        tool=TOOL_NAME,
        error_code=DOWNSTREAM_ERROR,
        message=f"GCS bucket {bucket!r} was not found or you do not
have access to it.",

```



```

        details={"bucket": bucket},
    )
    except GoogleCloudError as exc:
        logger.error("list_gcs_objects GCS error | bucket=%r | error=%s",
bucket, exc)
        return make_error(
            tool=TOOL_NAME,
            error_code=DOWNSTREAM_ERROR,
            message=f"GCS request failed: {exc}",
            details={"bucket": bucket, "exception_type":
type(exc).__name__},
        )
    except Exception as exc:
        logger.error("list_gcs_objects unexpected error | error=%s", exc)
        return make_error(
            tool=TOOL_NAME,
            error_code=DOWNSTREAM_ERROR,
            message=f"Unexpected error while listing GCS objects: {exc}",
            details={"exception_type": type(exc).__name__},
        )

```

tools/read_gcs_object.py

```

"""
tools/read_gcs_object.py

MCP tool: read_gcs_object

Reads the content of a single GCS object.
CRITICAL safety behaviour: checks `blob.size` (from metadata) BEFORE
downloading any content. Blobs exceeding GCS_MAX_FILE_SIZE_BYTES are
rejected before a single byte is transferred.

Returned success payload:
{
    "success": true,
    "tool": "read_gcs_object",
    "data": {
        "content": str,          (UTF-8 decoded; binary files are hex-
encoded)
        "encoding": "utf-8" | "hex",
        "size_bytes": int
    },
    "meta": {
        "bucket": str,
        "path": str,
        "content_type": str,
        "updated": str (ISO-8601)
    }
}

```

```

"""

from google.cloud import storage
from google.cloud.exceptions import GoogleCloudError, NotFound

from safety.gcs_safety import check_blob_size
from utils.error_format import DOWNSTREAM_ERROR, VALIDATION_ERROR,
make_error, make_success
from utils.logger import get_logger

logger = get_logger("tools.read_gcs_object")

TOOL_NAME = "read_gcs_object"

INPUT_SCHEMA = {
    "type": "object",
    "properties": {
        "bucket": {
            "type": "string",
            "description": "GCS bucket name (without gs:// prefix).",
            "minLength": 1,
        },
        "path": {
            "type": "string",
            "description": "Full object path within the bucket (e.g.
'data/report.csv').",
            "minLength": 1,
        },
    },
    "required": ["bucket", "path"],
    "additionalProperties": False,
}

def run(bucket: str, path: str) -> dict:
    """
    Reads a GCS object after a pre-download size safety check.

    Args:
        bucket: GCS bucket name.
        path: Object path within the bucket.

    Returns:
        A structured success or error envelope dict.
    """
    logger.info("read_gcs_object invoked | bucket=%r | path=%r", bucket,
path)

    # — Input validation (fail-fast)

    if not bucket or not bucket.strip():
        return make_error(
            tool=TOOL_NAME,
            error_code=VALIDATION_ERROR,

```

```

        message="'bucket' must be a non-empty string.",
        details={},
    )
    if not path or not path.strip():
        return make_error(
            tool=T00L_NAME,
            error_code=VALIDATION_ERROR,
            message="'path' must be a non-empty string.",
            details={},
        )

# — Metadata fetch + pre-download size check


---


    try:
        client = storage.Client()
        gcs_bucket = client.bucket(bucket)
        blob = gcs_bucket.get_blob(path) # Fetches metadata (size,
content_type, etc.)
    except GoogleCloudError as exc:
        logger.error(
            "read_gcs_object metadata fetch failed | bucket=%r | path=%r |
error=%s",
            bucket, path, exc,
        )
        return make_error(
            tool=T00L_NAME,
            error_code=DOWNSTREAM_ERROR,
            message=f"GCS metadata request failed: {exc}",
            details={"bucket": bucket, "path": path, "exception_type":
type(exc).__name__},
        )
    except Exception as exc:
        logger.error("read_gcs_object unexpected metadata error |
error=%s", exc)
        return make_error(
            tool=T00L_NAME,
            error_code=DOWNSTREAM_ERROR,
            message=f"Unexpected error fetching GCS metadata: {exc}",
            details={"exception_type": type(exc).__name__},
        )

    if blob is None:
        logger.warning(
            "read_gcs_object object not found | bucket=%r | path=%r",
            bucket, path
        )
        return make_error(
            tool=T00L_NAME,
            error_code=DOWNSTREAM_ERROR,
            message=f"Object gs://{bucket}/{path} does not exist.",
            details={"bucket": bucket, "path": path},
        )

    logger.debug(

```

```

        "read_gcs_object metadata | bucket=%r | path=%r | size=%d",
        bucket, path, blob.size,
    )

    # — Size safety check (fail-fast before any download)
    size_error = check_blob_size(blob.size, path)
    if size_error is not None:
        logger.warning(
            "read_gcs_object rejected by size gate | path=%r | size=%d",
            path, blob.size,
        )
        return size_error

    # — Content download (graceful degradation)
    try:
        raw_bytes = blob.download_as_bytes()
    except GoogleCloudError as exc:
        logger.error(
            "read_gcs_object download failed | bucket=%r | path=%r |
error=%s",
            bucket, path, exc,
        )
        return make_error(
            tool=T00L_NAME,
            error_code=DOWNSTREAM_ERROR,
            message=f"GCS download failed: {exc}",
            details={"bucket": bucket, "path": path, "exception_type":
type(exc).__name__},
        )
    except Exception as exc:
        logger.error("read_gcs_object unexpected download error |
error=%s", exc)
        return make_error(
            tool=T00L_NAME,
            error_code=DOWNSTREAM_ERROR,
            message=f"Unexpected error downloading GCS object: {exc}",
            details={"exception_type": type(exc).__name__},
        )

    # Attempt UTF-8 decode; fall back to hex for binary content
    try:
        content = raw_bytes.decode("utf-8")
        encoding = "utf-8"
    except UnicodeDecodeError:
        content = raw_bytes.hex()
        encoding = "hex"

    updated_iso = blob.updated.isoformat() if blob.updated else None

    logger.info(
        "read_gcs_object success | bucket=%r | path=%r | size=%d |
encoding=%s",

```

```

        bucket, path, blob.size, encoding,
    )

    return make_success(
        tool=TOOL_NAME,
        data={"content": content, "encoding": encoding, "size_bytes":
blob.size},
        meta={
            "bucket": bucket,
            "path": path,
            "content_type": blob.content_type,
            "updated": updated_iso,
        },
    )

```

tools/send_slack_message.py

```

"""
tools/send_slack_message.py

MCP tool: send_slack_message (STUBBED – no real Slack API call)

Demonstrates the safety-first pattern: the message passes through a real
in-memory token-bucket rate limiter BEFORE the (stubbed) send executes.
If the rate limit is exceeded, a structured safety error is returned.
If within limit, the message is logged and printed as if sent.

This stub makes it clear to client teams that:
1. Rate limiting must be implemented before shipping to production.
2. The real Slack API call is a one-line swap (replace the stub block).
3. The safety pattern (check → act) is identical to all other tools.

Returned success payload:
{
    "success": true,
    "tool": "send_slack_message",
    "data": {
        "channel": str,
        "message": str,
        "status": "stubbed_sent"
    },
    "meta": {
        "note": "Slack API call is stubbed. No real message was delivered."
    }
}
"""

from safety.slack_safety import check_slack_rate_limit
from utils.error_format import VALIDATION_ERROR, make_error, make_success
from utils.logger import get_logger

```

```

logger = get_logger("tools.send_slack_message")

TOOL_NAME = "send_slack_message"

INPUT_SCHEMA = {
    "type": "object",
    "properties": {
        "channel": {
            "type": "string",
            "description": "Slack channel name or ID (e.g. '#general' or 'C01234ABCDE').",
            "minLength": 1,
        },
        "message": {
            "type": "string",
            "description": "Message text to send.",
            "minLength": 1,
        },
    },
    "required": ["channel", "message"],
    "additionalProperties": False,
}

```

```

def run(channel: str, message: str) -> dict:
    """
    Validates, rate-checks, then stubs a Slack message send.

    Args:
        channel: Slack channel name or ID.
        message: Message content.

    Returns:
        A structured success or error envelope dict.
    """
    logger.info(
        "send_slack_message invoked | channel=%r | message_preview=%r",
        channel,
        message[:80],
    )

    # — Input validation (fail-fast)

```

```

    if not channel or not channel.strip():
        return make_error(
            tool=TOOL_NAME,
            error_code=VALIDATION_ERROR,
            message="'channel' must be a non-empty string.",
            details={},
        )
    if not message or not message.strip():
        return make_error(
            tool=TOOL_NAME,

```

```

        error_code=VALIDATION_ERROR,
        message="'message' must be a non-empty string.",
        details={},
    )

# — Rate limit check (fail-fast)


---


rate_error = check_slack_rate_limit(channel)
if rate_error is not None:
    logger.warning(
        "send_slack_message rate limit exceeded | channel=%r", channel
    )
    return rate_error

# — Stub send


---


# ⚠️ PRODUCTION TODO: Replace this block with a real Slack API call,
e.g.:
# import slack_sdk
# client = slack_sdk.WebClient(token=os.environ["SLACK_BOT_TOKEN"])
# resp = client.chat_postMessage(channel=channel, text=message)
formatted = (
    f"\n{' '*60}\n"
    f"[STUB] Slack message to {channel!r}\n"
    f"{' '*60}\n"
    f"{message}\n"
    f"{' '*60}\n"
)
print(formatted)
logger.info(
    "send_slack_message stubbed send | channel=%r | "
    "message_preview=%r",
    channel,
    message[:80],
)

return make_success(
    tool=TOOL_NAME,
    data={"channel": channel, "message": message, "status":
"stubbed_sent"},
    meta={"note": "Slack API call is stubbed. No real message was
delivered."},
)

```

server.py

```

"""
server.py

MCP server entry point.

```

Registers all four tools with the MCP SDK and starts the server using stdio transport (the correct transport for local / IDE integrations).

Tool registration pattern for each tool:

1. Describe the tool (name, description, input_schema).
2. In the call handler, extract and type-validate arguments.
3. Delegate to the tool module's run() function.
4. Return the structured envelope as a TextContent response.

The server itself does NO reasoning. It only exposes, validates, and executes.

"""

```
import asyncio
import json
import sys

import mcp.server.stdio
import mcp.types as types
from mcp.server import Server
from mcp.server.models import InitializationOptions

from tools import query_bigquery, list_gcs_objects, read_gcs_object,
send_slack_message
from utils.error_format import VALIDATION_ERROR, make_error
from utils.logger import get_logger

logger = get_logger("server")

app = Server("mcp-enterprise-gateway")

# — Tool Definitions

@app.list_tools()
async def list_tools() -> list[types.Tool]:
    logger.debug("list_tools called")
    return [
        types.Tool(
            name="query_bigquery",
            description=(
                "Execute a read-only BigQuery SELECT query. "
                "Non-SELECT statements and queries scanning more than "
                f"{100} MB are automatically rejected. "
                "Returns rows as structured JSON, schema metadata, job ID, "
                "bytes processed, and execution time."
            ),
            inputSchema=query_bigquery.INPUT_SCHEMA,
        ),
        types.Tool(
            name="list_gcs_objects",
```



```

        description=(
            "List objects in a Google Cloud Storage bucket under a
given prefix. "
            "Returns name, size, content-type, and last-updated
timestamp for each object."
        ),
        inputSchema=list_gcs_objects.INPUT_SCHEMA,
    ),
    types.Tool(
        name="read_gcs_object",
        description=(
            "Read the content of a single GCS object. "
            "Files larger than the configured maximum (default 10 MB)
are rejected "
            "before any content is downloaded. "
            "Returns UTF-8 text or hex-encoded content for binary
files."
        ),
        inputSchema=read_gcs_object.INPUT_SCHEMA,
    ),
    types.Tool(
        name="send_slack_message",
        description=(
            "Send a message to a Slack channel. "
            "STUBBED: logs and prints the message without calling the
real Slack API. "
            "A token-bucket rate limiter (default 5 messages / 60 s per
channel) "
            "is enforced regardless."
        ),
        inputSchema=send_slack_message.INPUT_SCHEMA,
    ),
]

```

— Tool Call Handler

```

@app.call_tool()
async def call_tool(
    name: str, arguments: dict
) -> list[types.TextContent | types.ImageContent | types.EmbeddedResource]:
    logger.info("call_tool dispatching | tool=%r | arguments=%r", name,
arguments)

    result: dict

    if name == "query_bigquery":
        sql = arguments.get("sql")
        if not isinstance(sql, str):
            result = make_error(
                tool="query_bigquery",
                error_code=VALIDATION_ERROR,
                message="Argument 'sql' is required and must be a string.",
            )

```

```
        details={"received": repr(sql)},
    )
    else:
        result = query_bigquery.run(sql=sql)

elif name == "list_gcs_objects":
    bucket = arguments.get("bucket")
    prefix = arguments.get("prefix", "")
    if not isinstance(bucket, str):
        result = make_error(
            tool="list_gcs_objects",
            error_code=VALIDATION_ERROR,
            message="Argument 'bucket' is required and must be a
string.",
            details={"received": repr(bucket)},
        )
    else:
        result = list_gcs_objects.run(bucket=bucket,
prefix=str(prefix))

elif name == "read_gcs_object":
    bucket = arguments.get("bucket")
    path = arguments.get("path")
    if not isinstance(bucket, str) or not isinstance(path, str):
        result = make_error(
            tool="read_gcs_object",
            error_code=VALIDATION_ERROR,
            message="Arguments 'bucket' and 'path' are required and
must be strings.",
            details={"bucket": repr(bucket), "path": repr(path)},
        )
    else:
        result = read_gcs_object.run(bucket=bucket, path=path)

elif name == "send_slack_message":
    channel = arguments.get("channel")
    message = arguments.get("message")
    if not isinstance(channel, str) or not isinstance(message, str):
        result = make_error(
            tool="send_slack_message",
            error_code=VALIDATION_ERROR,
            message="Arguments 'channel' and 'message' are required and
must be strings.",
            details={"channel": repr(channel), "message":
repr(message)},
        )
    else:
        result = send_slack_message.run(channel=channel,
message=message)

else:
    logger.error("call_tool unknown tool | name=%r", name)
    result = make_error(
        tool=name,
```

```

        error_code=VALIDATION_ERROR,
        message=f"Unknown tool {name!r}. Available tools:
query_bigquery, "
            "list_gcs_objects, read_gcs_object,
send_slack_message.",
        details={"requested_tool": name},
    )

    logger.debug(
        "call_tool response | tool=%r | success=%s", name,
        result.get("success")
    )

    return [types.TextContent(type="text", text=json.dumps(result,
indent=2))]

```

— Entry Point

```

async def main() -> None:
    logger.info("MCP Enterprise Gateway starting | transport=stdio")
    async with mcp.server.stdio.stdio_server() as (read_stream,
write_stream):
        logger.info("MCP server ready – waiting for client connections")
        await app.run(
            read_stream,
            write_stream,
            InitializationOptions(
                server_name="mcp-enterprise-gateway",
                server_version="1.0.0",
                capabilities=app.get_capabilities(
                    notification_options=None,
                    experimental_capabilities={},
                ),
            ),
        )

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        logger.info("MCP server shut down via KeyboardInterrupt")
        sys.exit(0)

```

tests/test_safety.py

```

"""
tests/test_safety.py

```

```

Tests for all three safety modules:
- sql_safety.validate_select_only
- gcs_safety.check_blob_size
- slack_safety.TokenBucketRateLimiter / check_slack_rate_limit
"""

import time
import pytest

from safety.sql_safety import validate_select_only
from safety.gcs_safety import check_blob_size
from safety.slack_safety import TokenBucketRateLimiter, reset_rate_limiter
from utils.error_format import VALIDATION_ERROR, SAFETY_LIMIT_EXCEEDED

# — SQL Safety


---



class TestSqlSafety:

    def test_valid_select_passes(self):
        result = validate_select_only("SELECT id, name FROM
`my_project.my_dataset.my_table`")
        assert result is None

    def test_select_with_where_passes(self):
        result = validate_select_only(
            "SELECT * FROM `project.dataset.table` WHERE created_at >
'2024-01-01'"
        )
        assert result is None

    def test_select_with_cte_passes(self):
        result = validate_select_only(
            "WITH cte AS (SELECT id FROM `project.dataset.table`) SELECT *
FROM cte"
        )
        assert result is None

    def test_select_with_limit_passes(self):
        result = validate_select_only(
            "SELECT col1, col2 FROM `project.dataset.table` LIMIT 100"
        )
        assert result is None

    def test_empty_sql_rejected(self):
        result = validate_select_only("")
        assert result is not None
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR

    def test_whitespace_only_rejected(self):
        result = validate_select_only(" \n\t ")
        assert result is not None

```

```
    assert result["error_code"] == VALIDATION_ERROR

def test_insert_rejected(self):
    result = validate_select_only(
        "INSERT INTO `project.dataset.table` (col) VALUES ('x')"
    )
    assert result is not None
    assert result["error_code"] == VALIDATION_ERROR
    assert "INSERT" in result["message"] or "INSERT" in
result["details"].get("detected_type", "")

def test_update_rejected(self):
    result = validate_select_only(
        "UPDATE `project.dataset.table` SET col = 1 WHERE id = 2"
    )
    assert result is not None
    assert result["error_code"] == VALIDATION_ERROR

def test_delete_rejected(self):
    result = validate_select_only("DELETE FROM `project.dataset.table`
WHERE id = 1")
    assert result is not None
    assert result["error_code"] == VALIDATION_ERROR

def test_drop_rejected(self):
    result = validate_select_only("DROP TABLE `project.dataset.table`")
    assert result is not None
    assert result["error_code"] == VALIDATION_ERROR

def test_create_rejected(self):
    result = validate_select_only(
        "CREATE TABLE `project.dataset.new_table` AS SELECT * FROM
`project.dataset.old`"
    )
    assert result is not None
    assert result["error_code"] == VALIDATION_ERROR

def test_multi_statement_rejected(self):
    result = validate_select_only(
        "SELECT 1; SELECT 2"
    )
    assert result is not None
    assert result["error_code"] == VALIDATION_ERROR
    assert "statement" in result["message"].lower()

def test_select_followed_by_drop_rejected(self):
    result = validate_select_only(
        "SELECT * FROM t; DROP TABLE t"
    )
    assert result is not None
    assert result["error_code"] == VALIDATION_ERROR

def test_dml_inside_cte_rejected(self):
    # CTE wrapping a DELETE – must be caught by the AST walk
```

```

        result = validate_select_only(
            "WITH cte AS (DELETE FROM `p.d.t` WHERE id = 1) SELECT 1"
        )
        # sqlglot may parse this differently across versions; at minimum it
must
        # not return None (i.e., it must not be silently allowed)
        # If sqlglot raises a ParseError or detects the DELETE node, we're
covered.
        # We accept either VALIDATION_ERROR outcome here.
        if result is not None:
            assert result["error_code"] == VALIDATION_ERROR

    def test_truncate_rejected(self):
        result = validate_select_only("TRUNCATE TABLE
`project.dataset.table`")
        assert result is not None
        assert result["error_code"] == VALIDATION_ERROR

    def test_error_envelope_shape(self):
        result = validate_select_only("DROP TABLE x")
        assert "success" in result
        assert "error_code" in result
        assert "message" in result
        assert "tool" in result
        assert "details" in result
        assert result["success"] is False

```

— GCS Safety

class TestGcsSafety:

```

    def test_within_limit_passes(self):
        # 1 byte under the 10MB default
        result = check_blob_size(10_485_759, "data/file.csv")
        assert result is None

    def test_exactly_at_limit_passes(self):
        result = check_blob_size(10_485_760, "data/file.csv")
        assert result is None

    def test_one_byte_over_limit_rejected(self):
        result = check_blob_size(10_485_761, "data/file.csv")
        assert result is not None
        assert result["success"] is False
        assert result["error_code"] == SAFETY_LIMIT_EXCEEDED

    def test_large_file_rejected(self):
        result = check_blob_size(500_000_000, "data/huge_dump.parquet")
        assert result is not None
        assert result["error_code"] == SAFETY_LIMIT_EXCEEDED
        assert result["details"]["file_size_bytes"] == 500_000_000

```

```
def test_error_contains_path(self):
    result = check_blob_size(999_999_999, "some/nested/path/file.bin")
    assert "some/nested/path/file.bin" in result["message"]

def test_error_envelope_shape(self):
    result = check_blob_size(999_999_999, "x.bin")
    assert "success" in result
    assert result["success"] is False
    assert "error_code" in result
    assert "tool" in result
    assert "details" in result
```

— Slack Rate Limiter

```
class TestTokenBucketRateLimiter:
```

```
    def setup_method(self):
        # Fresh limiter for each test: 5 calls per 60 seconds
        self.limiter = TokenBucketRateLimiter(max_calls=5,
period_seconds=60)

    def test_first_call_allowed(self):
        result = self.limiter.check_and_consume("general")
        assert result is None

    def test_five_calls_allowed(self):
        for _ in range(5):
            result = self.limiter.check_and_consume("general")
            assert result is None

    def test_sixth_call_rejected(self):
        for _ in range(5):
            self.limiter.check_and_consume("general")
        result = self.limiter.check_and_consume("general")
        assert result is not None
        assert result["success"] is False
        assert result["error_code"] == SAFETY_LIMIT_EXCEEDED

    def test_different_channels_are_independent(self):
        for _ in range(5):
            self.limiter.check_and_consume("channel-a")
        # channel-a is exhausted; channel-b should still work
        result = self.limiter.check_and_consume("channel-b")
        assert result is None

    def test_tokens_refill_over_time(self):
        # Use a fast limiter: 2 calls per 0.2 seconds (rate = 10/s)
        fast_limiter = TokenBucketRateLimiter(max_calls=2,
period_seconds=1)
        fast_limiter.check_and_consume("ch")
        fast_limiter.check_and_consume("ch")
        # Bucket is empty; third call should fail
```

```

        result = fast_limiter.check_and_consume("ch")
        assert result is not None
        # Wait 1.1 seconds for full refill
        time.sleep(1.1)
        result = fast_limiter.check_and_consume("ch")
        assert result is None # Refilled

def test_reset_clears_channel(self):
    for _ in range(5):
        self.limiter.check_and_consume("general")
    self.limiter.reset("general")
    result = self.limiter.check_and_consume("general")
    assert result is None

def test_reset_all_clears_all_channels(self):
    for _ in range(5):
        self.limiter.check_and_consume("ch1")
    for _ in range(5):
        self.limiter.check_and_consume("ch2")
    self.limiter.reset()
    assert self.limiter.check_and_consume("ch1") is None
    assert self.limiter.check_and_consume("ch2") is None

def test_error_envelope_shape(self):
    for _ in range(5):
        self.limiter.check_and_consume("ch")
    result = self.limiter.check_and_consume("ch")
    assert "success" in result
    assert result["success"] is False
    assert "error_code" in result
    assert "tool" in result
    assert "details" in result
    assert "retry_after_seconds" in result["details"]

def test_module_level_reset(self):
    # Ensure the module-level reset helper works for test isolation
    reset_rate_limiter("general")
    from safety.slack_safety import check_slack_rate_limit
    result = check_slack_rate_limit("general")
    assert result is None

```

tests/test_query_bigquery.py

```

"""
tests/test_query_bigquery.py

Tests for tools/query_bigquery.py
All BigQuery API calls are mocked – no real network calls occur.
"""

```



```

import pytest
from unittest.mock import MagicMock, patch, PropertyMock

from tools.query_bigquery import run, TOOL_NAME
from utils.error_format import VALIDATION_ERROR, SAFETY_LIMIT_EXCEEDED,
DOWNSTREAM_ERROR

# Helper: build a mock QueryJob that looks like a dry-run result
def _make_dry_run_job(total_bytes: int):
    job = MagicMock()
    type(job).total_bytes_processed =
PropertyMock(return_value=total_bytes)
    return job

# Helper: build a mock QueryJob for an execution result
def _make_exec_job(rows, schema_fields=None, bytes_processed=1024,
job_id="job-abc-123"):
    job = MagicMock()
    job.job_id = job_id
    type(job).total_bytes_processed =
PropertyMock(return_value=bytes_processed)
    type(job).schema = PropertyMock(return_value=schema_fields or [])
    job.result.return_value = rows
    return job

class TestQueryBigqueryValidation:

    def test_empty_sql_returns_validation_error(self):
        result = run(sql="")
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR
        assert result["tool"] == TOOL_NAME

    def test_insert_sql_rejected(self):
        result = run(sql="INSERT INTO t (col) VALUES (1)")
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR

    def test_delete_sql_rejected(self):
        result = run(sql="DELETE FROM t WHERE id = 1")
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR

    def test_drop_sql_rejected(self):
        result = run(sql="DROP TABLE t")
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR

    def test_multi_statement_rejected(self):
        result = run(sql="SELECT 1; SELECT 2")
        assert result["success"] is False

```

```

        assert result["error_code"] == VALIDATION_ERROR

class TestQueryBigqueryCostGate:

    @patch("tools.query_bigquery.bigquery.Client")
    def test_exceeding_byte_limit_rejected(self, mock_client_cls):
        mock_client = mock_client_cls.return_value
        # Dry-run returns 200MB – over the 100MB limit
        mock_client.query.return_value =
        _make_dry_run_job(total_bytes=209_715_200)

        result = run(sql="SELECT * FROM `p.d.t`")
        assert result["success"] is False
        assert result["error_code"] == SAFETY_LIMIT_EXCEEDED
        assert "estimated_bytes" in result["details"]
        assert result["details"]["estimated_bytes"] == 209_715_200

    @patch("tools.query_bigquery.bigquery.Client")
    def test_within_byte_limit_proceeds(self, mock_client_cls):
        mock_client = mock_client_cls.return_value
        dry_run_job = _make_dry_run_job(total_bytes=50_000)
        exec_job = _make_exec_job(rows=[], bytes_processed=50_000)

        # First call (dry-run) returns dry_run_job; second call (execute)
returns exec_job
        mock_client.query.side_effect = [dry_run_job, exec_job]
        mock_client.project = "test-project"

        result = run(sql="SELECT id FROM `p.d.t` LIMIT 10")
        assert result["success"] is True
        assert result["tool"] == TOOL_NAME

    @patch("tools.query_bigquery.bigquery.Client")
    def test_dry_run_google_cloud_error_returns_downstream_error(self,
mock_client_cls):
        from google.cloud.exceptions import GoogleCloudError
        mock_client = mock_client_cls.return_value
        mock_client.query.side_effect = GoogleCloudError("quota exceeded")

        result = run(sql="SELECT 1")
        assert result["success"] is False
        assert result["error_code"] == DOWNSTREAM_ERROR

class TestQueryBigqueryExecution:

    @patch("tools.query_bigquery.bigquery.Client")
    def test_success_returns_rows_and_meta(self, mock_client_cls):
        mock_client = mock_client_cls.return_value
        mock_client.project = "my-project"

        dry_run_job = _make_dry_run_job(total_bytes=1024)

```

```

# Build mock rows
row1 = MagicMock()
row1.items.return_value = [("id", 1), ("name", "Alice")]
row2 = MagicMock()
row2.items.return_value = [("id", 2), ("name", "Bob")]

# Build mock schema
field1 = MagicMock()
field1.name = "id"
field1.field_type = "INTEGER"
field1.mode = "NULLABLE"
field2 = MagicMock()
field2.name = "name"
field2.field_type = "STRING"
field2.mode = "NULLABLE"

exec_job = _make_exec_job(
    rows=[row1, row2],
    schema_fields=[field1, field2],
    bytes_processed=1024,
    job_id="job-xyz",
)

mock_client.query.side_effect = [dry_run_job, exec_job]

result = run(sql="SELECT id, name FROM `p.d.users` LIMIT 2")
assert result["success"] is True
assert result["data"]["row_count"] == 2
assert result["data"]["rows"][0] == {"id": 1, "name": "Alice"}
assert result["data"]["rows"][1] == {"id": 2, "name": "Bob"}
assert len(result["data"]["schema"]) == 2
assert result["meta"]["job_id"] == "job-xyz"
assert result["meta"]["bytes_processed"] == 1024

@patch("tools.query_bigquery.bigquery.Client")
def test_execution_google_cloud_error_returns_downstream_error(self,
mock_client_cls):
    from google.cloud.exceptions import GoogleCloudError
    mock_client = mock_client_cls.return_value
    mock_client.project = "my-project"

    dry_run_job = _make_dry_run_job(total_bytes=1024)
    exec_job = MagicMock()
    exec_job.result.side_effect = GoogleCloudError("backend error")

    mock_client.query.side_effect = [dry_run_job, exec_job]

    result = run(sql="SELECT 1")
    assert result["success"] is False
    assert result["error_code"] == DOWNSTREAM_ERROR

```

tests/test_list_gcs_objects.py

```
"""
```

```
tests/test_list_gcs_objects.py
```

```
Tests for tools/list_gcs_objects.py
```

```
All GCS API calls are mocked.
```

```
"""
```

```
import pytest
```

```
from datetime import datetime, timezone
```

```
from unittest.mock import MagicMock, patch
```

```
from tools.list_gcs_objects import run, TOOL_NAME
```

```
from utils.error_format import VALIDATION_ERROR, DOWNSTREAM_ERROR
```

```
def _make_blob(name: str, size: int, content_type: str, updated: datetime |  
None):
```

```
    blob = MagicMock()
```

```
    blob.name = name
```

```
    blob.size = size
```

```
    blob.content_type = content_type
```

```
    blob.updated = updated
```

```
    return blob
```

```
class TestListGcs0bjects:
```

```
    def test_empty_bucket_name_rejected(self):
```

```
        result = run(bucket="", prefix="")
```

```
        assert result["success"] is False
```

```
        assert result["error_code"] == VALIDATION_ERROR
```

```
    @patch("tools.list_gcs_objects.storage.Client")
```

```
    def test_returns_object_list_on_success(self, mock_client_cls):
```

```
        mock_client = mock_client_cls.return_value
```

```
        updated = datetime(2024, 6, 1, 12, 0, 0, tzinfo=timezone.utc)
```

```
        mock_client.list_blobs.return_value = [
```

```
            _make_blob("data/file1.csv", 1024, "text/csv", updated),
```

```
            _make_blob("data/file2.json", 2048, "application/json",
```

```
updated),
```

```
        ]
```

```
        result = run(bucket="my-bucket", prefix="data/")
```

```
        assert result["success"] is True
```

```
        assert result["tool"] == TOOL_NAME
```

```
        assert result["data"]["object_count"] == 2
```

```
        assert result["data"]["objects"][0]["name"] == "data/file1.csv"
```

```
        assert result["data"]["objects"][0]["size_bytes"] == 1024
```

```
        assert result["data"]["objects"][0]["content_type"] == "text/csv"
```

```
        assert "2024-06-01" in result["data"]["objects"][0]["updated"]
```

```

@patch("tools.list_gcs_objects.storage.Client")
def test_empty_prefix_lists_all_objects(self, mock_client_cls):
    mock_client = mock_client_cls.return_value
    mock_client.list_blobs.return_value = [
        _make_blob("root_file.txt", 100, "text/plain", None),
    ]
    result = run(bucket="my-bucket", prefix="")
    assert result["success"] is True
    assert result["data"]["object_count"] == 1

@patch("tools.list_gcs_objects.storage.Client")
def test_not_found_returns_downstream_error(self, mock_client_cls):
    from google.cloud.exceptions import NotFound
    mock_client = mock_client_cls.return_value
    mock_client.list_blobs.side_effect = NotFound("bucket not found")

    result = run(bucket="nonexistent-bucket", prefix="")
    assert result["success"] is False
    assert result["error_code"] == DOWNSTREAM_ERROR
    assert "not found" in result["message"].lower()

@patch("tools.list_gcs_objects.storage.Client")
def test_gcs_error_returns_downstream_error(self, mock_client_cls):
    from google.cloud.exceptions import GoogleCloudError
    mock_client = mock_client_cls.return_value
    mock_client.list_blobs.side_effect = GoogleCloudError("network
error")

    result = run(bucket="my-bucket", prefix="")
    assert result["success"] is False
    assert result["error_code"] == DOWNSTREAM_ERROR

@patch("tools.list_gcs_objects.storage.Client")
def test_empty_bucket_returns_zero_objects(self, mock_client_cls):
    mock_client = mock_client_cls.return_value
    mock_client.list_blobs.return_value = []

    result = run(bucket="empty-bucket", prefix="")
    assert result["success"] is True
    assert result["data"]["object_count"] == 0
    assert result["data"]["objects"] == []

```

tests/test_read_gcs_object.py

```

"""
tests/test_read_gcs_object.py

Tests for tools/read_gcs_object.py
All GCS API calls are mocked – no real network calls.

```

```

"""

import pytest
from unittest.mock import MagicMock, patch, PropertyMock
from datetime import datetime, timezone

from tools.read_gcs_object import run, TOOL_NAME
from utils.error_format import VALIDATION_ERROR, SAFETY_LIMIT_EXCEEDED,
DOWNSTREAM_ERROR

def _make_blob(
    size: int,
    content: bytes,
    content_type: str = "text/plain",
    updated: datetime | None = None,
):
    blob = MagicMock()
    blob.size = size
    blob.content_type = content_type
    blob.updated = updated or datetime(2024, 1, 1, tzinfo=timezone.utc)
    blob.download_as_bytes.return_value = content
    return blob

class TestReadGcsObjectValidation:

    def test_empty_bucket_rejected(self):
        result = run(bucket="", path="file.txt")
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR

    def test_empty_path_rejected(self):
        result = run(bucket="my-bucket", path="")
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR

class TestReadGcsObjectSizeGate:

    @patch("tools.read_gcs_object.storage.Client")
    def test_file_over_limit_rejected_before_download(self,
mock_client_cls):
        mock_client = mock_client_cls.return_value
        # 11 MB blob – over the 10 MB default limit
        blob = _make_blob(size=11_534_336, content=b"")
        mock_client.bucket.return_value.get_blob.return_value = blob

        result = run(bucket="my-bucket", path="large_file.parquet")
        assert result["success"] is False
        assert result["error_code"] == SAFETY_LIMIT_EXCEEDED
        # Content download must NOT have been called
        blob.download_as_bytes.assert_not_called()

```

```

@patch("tools.read_gcs_object.storage.Client")
def test_file_within_limit_is_downloaded(self, mock_client_cls):
    mock_client = mock_client_cls.return_value
    blob = _make_blob(size=500, content=b"hello world")
    mock_client.bucket.return_value.get_blob.return_value = blob

    result = run(bucket="my-bucket", path="small.txt")
    assert result["success"] is True
    assert result["data"]["content"] == "hello world"
    assert result["data"]["encoding"] == "utf-8"
    assert result["data"]["size_bytes"] == 500

```

```

class TestReadGcsObjectContent:

```

```

    @patch("tools.read_gcs_object.storage.Client")
    def test_utf8_file_returned_as_text(self, mock_client_cls):
        mock_client = mock_client_cls.return_value
        content = "id,name\n1,Alice\n2,Bob\n".encode("utf-8")
        blob = _make_blob(size=len(content), content=content,
content_type="text/csv")
        mock_client.bucket.return_value.get_blob.return_value = blob

        result = run(bucket="my-bucket", path="data.csv")
        assert result["success"] is True
        assert result["data"]["encoding"] == "utf-8"
        assert "Alice" in result["data"]["content"]
        assert result["meta"]["content_type"] == "text/csv"

    @patch("tools.read_gcs_object.storage.Client")
    def test_binary_file_returned_as_hex(self, mock_client_cls):
        mock_client = mock_client_cls.return_value
        content = bytes([0x89, 0x50, 0x4E, 0x47]) # PNG header bytes
        blob = _make_blob(size=4, content=content,
content_type="image/png")
        mock_client.bucket.return_value.get_blob.return_value = blob

        result = run(bucket="my-bucket", path="image.png")
        assert result["success"] is True
        assert result["data"]["encoding"] == "hex"
        assert result["data"]["content"] == "89504e47"

    @patch("tools.read_gcs_object.storage.Client")
    def test_object_not_found_returns_downstream_error(self,
mock_client_cls):
        mock_client = mock_client_cls.return_value
        mock_client.bucket.return_value.get_blob.return_value = None

        result = run(bucket="my-bucket", path="missing.txt")
        assert result["success"] is False
        assert result["error_code"] == DOWNSTREAM_ERROR
        assert "does not exist" in result["message"]

    @patch("tools.read_gcs_object.storage.Client")

```

```

def test_download_error_returns_downstream_error(self,
mock_client_cls):
    from google.cloud.exceptions import GoogleCloudError
    mock_client = mock_client_cls.return_value
    blob = _make_blob(size=100, content=b'')
    blob.download_as_bytes.side_effect = GoogleCloudError("download
interrupted")
    mock_client.bucket.return_value.get_blob.return_value = blob

    result = run(bucket="my-bucket", path="flaky.txt")
    assert result["success"] is False
    assert result["error_code"] == DOWNSTREAM_ERROR

```

tests/test_send_slack_message.py

```

"""
tests/test_send_slack_message.py

Tests for tools/send_slack_message.py
Rate limiter state is reset between tests for isolation.
"""

import pytest

from tools.send_slack_message import run, TOOL_NAME
from safety.slack_safety import reset_rate_limiter
from utils.error_format import VALIDATION_ERROR, SAFETY_LIMIT_EXCEEDED

class TestSendSlackMessage:

    def setup_method(self):
        # Reset rate limiter state before each test
        reset_rate_limiter()

    def test_empty_channel_rejected(self):
        result = run(channel="", message="Hello")
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR

    def test_empty_message_rejected(self):
        result = run(channel="#general", message="")
        assert result["success"] is False
        assert result["error_code"] == VALIDATION_ERROR

    def test_valid_message_succeeds(self):
        result = run(channel="#general", message="Hello from MCP!")
        assert result["success"] is True
        assert result["tool"] == TOOL_NAME
        assert result["data"]["status"] == "stubbed_sent"

```



```

    assert result["data"]["channel"] == "#general"
    assert result["data"]["message"] == "Hello from MCP!"

def test_rate_limit_allows_five_messages(self):
    for i in range(5):
        result = run(channel="#alerts", message=f"Message {i}")
        assert result["success"] is True, f"Message {i} should succeed"

def test_rate_limit_rejects_sixth_message(self):
    for i in range(5):
        run(channel="#alerts", message=f"Message {i}")
    result = run(channel="#alerts", message="Message 6")
    assert result["success"] is False
    assert result["error_code"] == SAFETY_LIMIT_EXCEEDED

def test_rate_limit_is_per_channel(self):
    for i in range(5):
        run(channel="#channel-a", message=f"Msg {i}")
    # channel-a is exhausted; channel-b should still accept messages
    result = run(channel="#channel-b", message="Hello channel-b")
    assert result["success"] is True

def test_success_envelope_shape(self):
    result = run(channel="#test", message="Test message")
    assert "success" in result
    assert "tool" in result
    assert "data" in result
    assert "meta" in result
    assert result["success"] is True

def test_error_envelope_shape_on_rate_limit(self):
    for i in range(5):
        run(channel="#flood", message=f"Msg {i}")
    result = run(channel="#flood", message="One too many")
    assert "success" in result
    assert "error_code" in result
    assert "message" in result
    assert "tool" in result
    assert "details" in result
    assert result["success"] is False

def test_stub_note_in_meta(self):
    result = run(channel="#general", message="Test")
    assert "stubbed" in result["meta"]["note"].lower()

```

tests/__init__.py

```
# tests/__init__.py – marks tests as a package
```

test_client.py (Optional / Stretch Goal)

```

"""
test_client.py - OPTIONAL / STRETCH GOAL
=====

A minimal MCP client that:
1. Spawns server.py as a subprocess over stdio.
2. Connects using the official MCP Python SDK's ClientSession.
3. Retrieves the server's tool catalogue.
4. Sends a natural-language instruction to the Groq API
   (llama-3.3-70b-versatile) with the tool schemas attached.
5. Lets the model decide which tool to call and with what arguments.
6. Executes that tool call via the MCP session.
7. Prints the structured result.

Why does the LLM call belong HERE and not in server.py?
- The server is a tool executor, not a reasoner. Mixing LLM calls into
  the server would break the MCP client-server separation and make the
  server stateful in a way that is hard to audit.
- The client is the appropriate place for reasoning. This file
demonstrates
  that pattern explicitly.

Prerequisites:
- pip install groq mcp
- GROQ_API_KEY set in .env (see .env.example)
- server.py must be present and functional

Usage:
python test_client.py "List objects in my-bucket under prefix data/"
"""

import asyncio
import json
import os
import sys
from pathlib import Path

import groq
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

# Load .env from the project root
from dotenv import load_dotenv
load_dotenv(Path(__file__).parent / ".env")

GROQ_API_KEY = os.environ.get("GROQ_API_KEY")
if not GROQ_API_KEY:
    print(
        "ERROR: GROQ_API_KEY is not set. Add it to your .env file and re-
run.",

```

```

        file=sys.stderr,
    )
    sys.exit(1)

MODEL = "llama-3.3-70b-versatile"

def _mcp_tool_to_groq_tool(tool) -> dict:
    """
    Converts an MCP Tool object to the Groq/OpenAI function-calling schema
    format.
    """
    return {
        "type": "function",
        "function": {
            "name": tool.name,
            "description": tool.description or "",
            "parameters": tool.inputSchema,
        },
    }

async def run(instruction: str) -> None:
    """
    Connects to server.py via stdio, asks Groq which tool to call,
    executes it, and prints the result.
    """
    server_params = StdioServerParameters(
        command=sys.executable,
        args=[str(Path(__file__).parent / "server.py")],
        env=None,
    )

    groq_client = groq.Groq(api_key=GROQ_API_KEY)

    print(f"\n[test_client] Instruction: {instruction!r}\n")

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            print("[test_client] Connected to MCP server")

            # 1. Fetch the tool catalogue from the server
            tools_response = await session.list_tools()
            tools = tools_response.tools
            print(f"[test_client] Available tools: {[t.name for t in tools]}")

            groq_tools = [_mcp_tool_to_groq_tool(t) for t in tools]

            # 2. Ask Groq which tool to call
            messages = [
                {
                    "role": "system",

```

```

        "content": (
            "You are an assistant that calls tools to answer
user questions. "
            "You have access to BigQuery, Google Cloud Storage,
and Slack tools. "
            "Call exactly one tool based on the user's
instruction. "
            "Do not guess data – only use what the tool
returns."
        ),
    },
    {"role": "user", "content": instruction},
]

response = groq_client.chat.completions.create(
    model=MODEL,
    messages=messages,
    tools=groq_tools,
    tool_choice="auto",
)

choice = response.choices[0]

if choice.finish_reason != "tool_calls" or not
choice.message.tool_calls:
    print(
        "[test_client] Model did not request a tool call.\n"
        f"Model response: {choice.message.content}"
    )
    return

tool_call = choice.message.tool_calls[0]
tool_name = tool_call.function.name
tool_args = json.loads(tool_call.function.arguments)

print(f"\n[test_client] Model selected tool: {tool_name!r}")
print(f"[test_client] Arguments: {json.dumps(tool_args,
indent=2)}")

# 3. Execute the tool call via the MCP session
result = await session.call_tool(tool_name, tool_args)

# 4. Parse and display the result
if result.content:
    raw_text = result.content[0].text
    try:
        parsed = json.loads(raw_text)
        print(f"\n[test_client] Tool
result:\n{json.dumps(parsed, indent=2)}")
    except json.JSONDecodeError:
        print(f"\n[test_client] Tool result
(raw):\n{raw_text}")
    else:
        print("[test_client] Tool returned no content.")

```

```

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(
            "Usage: python test_client.py \"<natural language\ninstruction>\"\n"
            "Example: python test_client.py \"List objects in my-bucket\nunder prefix data/\""
        )
        sys.exit(1)

    instruction = " ".join(sys.argv[1:])
    asyncio.run(run(instruction))

```

4. Code Logic & Deep-Dive

How `server.py` Wires Everything Together

`server.py` constructs a single `Server` instance from the MCP SDK and attaches two async handlers via decorators: `@app.list_tools()` and `@app.call_tool()`.

`@app.list_tools()` is called by any MCP client when it first connects, or whenever it needs to refresh its knowledge of what the server can do. It returns a list of `types.Tool` objects, each containing a `name`, a `description`, and an `inputSchema` (a JSON Schema dict). These schemas are defined in each tool module and imported into `server.py` — they are the single source of truth for what arguments each tool accepts. The MCP client uses these schemas both for display (showing the user available tools) and to validate arguments before sending a call.

`@app.call_tool()` receives `name` (tool name as a string) and `arguments` (a raw dict from the client). The handler performs a type check on the raw arguments — verifying that required keys are present and are the expected types — then delegates to the tool module's `run()` function. The result, which is always a structured dict (either a success or error envelope), is serialised to JSON and returned as a `TextContent` block. The MCP SDK transmits this back to the client over stdio.

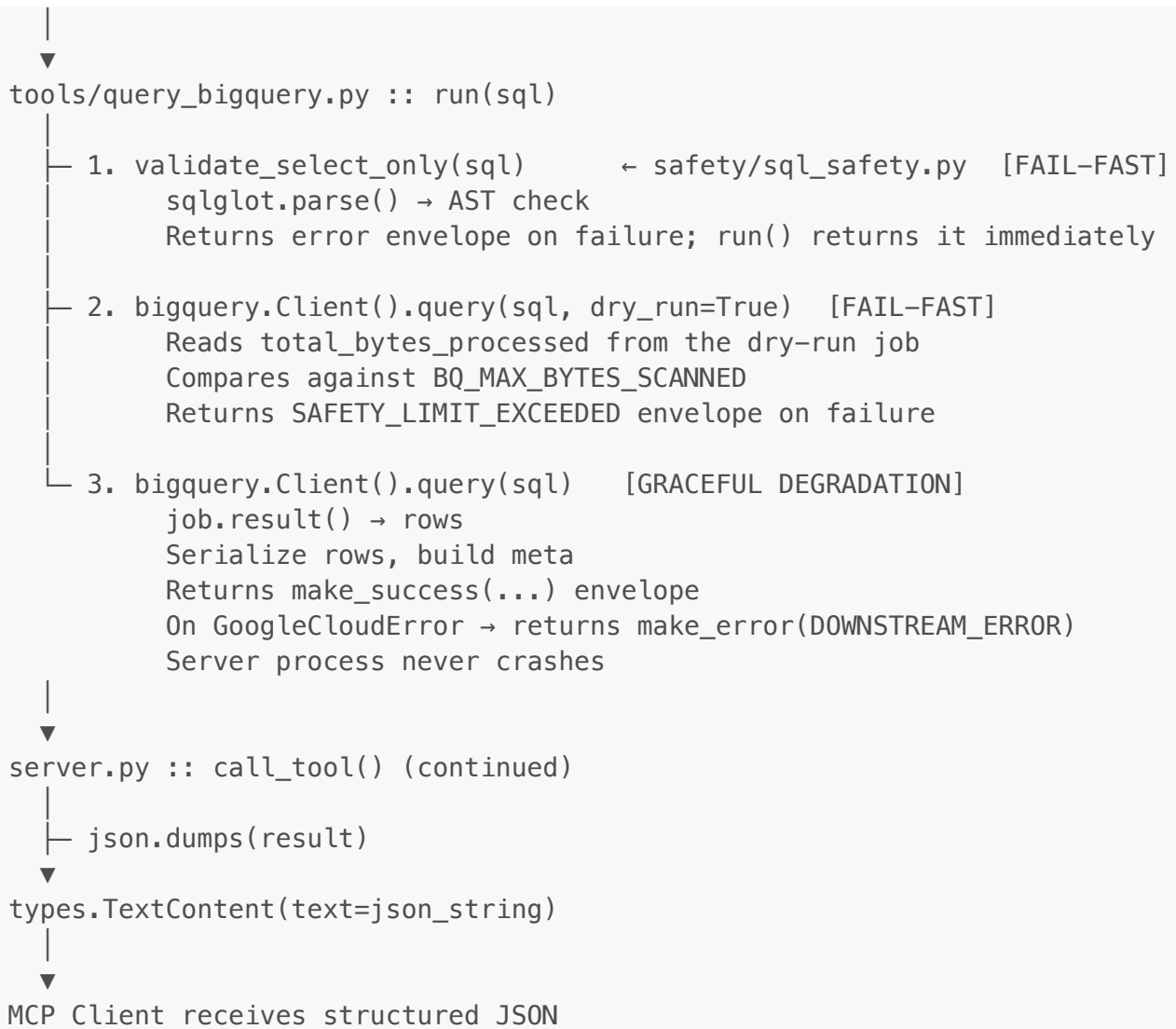
The `main()` coroutine opens a stdio server context using `mcp.server.stdio.stdio_server()`, which yields the `(read_stream, write_stream)` pair that the SDK uses to communicate with the client process. `app.run()` blocks until the client disconnects, handling all JSON-RPC framing internally.

Request Flow: End to End

```

MCP Client
|
|  JSON-RPC call_tool {name: "query_bigquery", arguments: {sql: "SELECT
..."}}
▼
server.py :: call_tool()
|
|  Type-check arguments (fail-fast, before any module is called)

```



BigQuery Dry-Run: How It Works and Its Limitations

When you call `bigquery.Client().query(sql, job_config=QueryJobConfig(dry_run=True, use_query_cache=False))`, BigQuery does **not** execute the query. It parses the SQL, resolves table references, and computes the estimated number of bytes the query would scan if executed. This estimate is returned synchronously in `job.total_bytes_processed`.

Limitations to know:

- **It is an estimate.** For queries over partitioned tables, the actual bytes scanned after applying partition pruning may be lower than the dry-run estimate. For highly dynamic queries, it could be higher.
- **It does not catch all errors.** A query that would fail at execution time (e.g., a function called with wrong argument types in some edge cases) may still pass the dry-run successfully.
- **Cached results are excluded.** `use_query_cache=False` ensures we measure the true scan volume, not zero bytes from a cache hit.
- **Free-tier tables.** Some public datasets are scanned without charge; the byte estimate is still correct, but the billing impact is zero.

GCS Size Pre-Check: Why Metadata First

`storage.Client().bucket(name).get_blob(path)` issues an HTTP GET request to the GCS metadata endpoint (equivalent to `gsutil stat`). This populates `blob.size`, `blob.content_type`, `blob.updated`, etc., **without transferring the object's content**. Only after the `check_blob_size()` safety gate passes does the code call `blob.download_as_bytes()`, which performs the actual content transfer.

The alternative — downloading first, then checking length — would:

1. Consume memory proportional to the file size before rejecting it.
2. Waste egress bandwidth and incur egress costs.
3. Risk OOM errors on the server process for unexpectedly large files.

By checking `blob.size` from metadata, files can be rejected in milliseconds with zero memory allocation for content.

5. Deployment & Execution Guide

Step 1: Scaffold the Project

```
# Navigate to where you want the project to live
cd ~/projects # or wherever you keep code

# Run the scaffold script
chmod +x setup.sh
./setup.sh
```

This creates all directories and empty stub files, then creates the `myenv` virtual environment inside `mcp_server/`.

Step 2: Activate the Virtual Environment and Install Dependencies

```
cd mcp_server
source myenv/bin/activate

# Verify you're using the virtualenv Python
which python3
# Expected: /path/to/mcp_server/myenv/bin/python3

pip install --upgrade pip
pip install -r requirements.txt
```

Step 3: Copy All Implementation Code

At this point the stub files exist but are empty. Copy every code block from Section 3 of this guide into the corresponding file. The files and their locations are:

File	Content from Section 3
utils/logger.py	Logger configuration
utils/error_format.py	Error/success envelope builders
config/settings.py	Environment variable loader
safety/sql_safety.py	sqlglot-based SQL validator
safety/gcs_safety.py	GCS size pre-check
safety/slack_safety.py	Token-bucket rate limiter
tools/query_bigquery.py	BigQuery tool
tools/list_gcs_objects.py	GCS list tool
tools/read_gcs_object.py	GCS read tool
tools/send_slack_message.py	Slack stub tool
server.py	MCP server entry point
tests/test_safety.py	Safety module tests
tests/test_query_bigquery.py	BigQuery tool tests
tests/test_list_gcs_objects.py	GCS list tool tests
tests/test_read_gcs_object.py	GCS read tool tests
tests/test_send_slack_message.py	Slack tool tests
test_client.py	Optional Groq client

Step 4: Configure Environment Variables

```
cp .env.example .env
```

Open `.env` in your editor and fill in the values:

```
# Set to your GCP project for BigQuery – omit if using default from gcloud
BQ_PROJECT_ID=my-gcp-project-id

# These defaults are fine for most use cases
BQ_MAX_BYTES_SCANNED=104857600
GCS_MAX_FILE_SIZE_BYTES=10485760
SLACK_RATE_LIMIT_MAX_CALLS=5
SLACK_RATE_LIMIT_PERIOD_SECONDS=60

# Optional: path to a service account JSON key
# Leave commented out if using gcloud auth application-default login
# GOOGLE_APPLICATION_CREDENTIALS=/Users/yourname/.config/gcloud/my-sa-
```



```
key.json

# Only needed for test_client.py
# GROQ_API_KEY=gsk_...
```

Step 5: Authenticate with Google Cloud

For local development, Application Default Credentials (ADC) are the correct approach:

```
# Install the Google Cloud CLI if not already installed:
# brew install --cask google-cloud-sdk

# Log in and set application default credentials
gcloud auth application-default login

# Set a default project (optional but recommended)
gcloud config set project my-gcp-project-id
```

This writes credentials to `~/.config/gcloud/application_default_credentials.json`. The BigQuery and GCS clients pick these up automatically. If you need to use a service account (e.g., for CI/CD), set `GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa-key.json` in `.env` instead.

Step 6: Run the Test Suite

```
# Ensure virtualenv is active
source myenv/bin/activate

# Run all tests with verbose output
pytest -v tests/

# Expected output (all tests should pass; no real API calls are made):
# tests/test_safety.py::TestSqlSafety::test_valid_select_passes PASSED
# tests/test_safety.py::TestSqlSafety::test_insert_rejected PASSED
# ... (all tests) ...
# ===== N passed in X.XXs
=====
```

If any tests fail, check that all files were saved correctly and that the virtualenv is active.

Step 7: Launch the MCP Server

```
# Ensure virtualenv is active
source myenv/bin/activate

# Start the server (stdio mode – it will wait for a client to connect)
python server.py
```

You should see log output like:

```
2024-06-01 12:00:00 | INFO | MCP Enterprise Gateway starting |
transport=stdio
2024-06-01 12:00:00 | INFO | MCP server ready – waiting for client
connections
```

The server stays alive waiting for a client. To connect Claude Desktop or Cursor, add an entry to their MCP config JSON pointing to this `server.py` with the full path to the `myenv` Python interpreter.

Example Claude Desktop config (`~/Library/Application Support/Claude/claude_desktop_config.json`):

```
{
  "mcpServers": {
    "enterprise-gateway": {
      "command": "/Users/yourname/projects/mcp_server/myenv/bin/python3",
      "args": ["/Users/yourname/projects/mcp_server/server.py"]
    }
  }
}
```

Step 8: Verify Logging

After running any tool call, inspect `output.log`:

```
tail -f output.log
```

Expected log lines for a successful BigQuery query:

```
2024-06-01 12:05:00 | INFO | mcp_server.server | call_tool dispatching |
tool='query_bigquery' | arguments={'sql': 'SELECT ...'}
2024-06-01 12:05:00 | INFO | mcp_server.tools.query_bigquery |
query_bigquery invoked | sql_preview='SELECT ...'
2024-06-01 12:05:00 | DEBUG | mcp_server.tools.query_bigquery |
query_bigquery dry-run complete | estimated_bytes=4096 | limit=104857600
2024-06-01 12:05:00 | INFO | mcp_server.tools.query_bigquery |
```

```
query_bigquery success | job_id=job-abc | rows=10 | bytes_processed=4096 |  
elapsed_ms=342.1  
2024-06-01 12:05:00 | DEBUG | mcp_server.server | call_tool response |  
tool='query_bigquery' | success=True
```

Step 9: Optional — Run the Groq Test Client

```
# Ensure GROQ_API_KEY is in .env, then:  
python test_client.py "List the objects in my-data-bucket under the prefix  
reports/2024/"
```

The client spawns `server.py` as a subprocess, connects to it, asks Groq's model which tool to use, calls it, and prints the result.

Deliverable: README.md

MCP Enterprise Gateway

A production-grade MCP (Model Context Protocol) server providing governed access to BigQuery, Google Cloud Storage, and Slack for any MCP-compatible LLM client.

Quick Start

```
```bash  
cd mcp_server
source myenv/bin/activate
python server.py
```

See [guide.md](#) Section 5 for full setup instructions.

---

## Tool Catalogue

### 1. query\_bigquery

Executes a read-only BigQuery SELECT query with SQL validation and cost gating.

#### Input Schema

```
{
 "type": "object",
 "properties": {
 "sql": {
 "type": "string",
```

```
 "description": "A single, read-only BigQuery SELECT statement.",
 "minLength": 1
 },
 "required": ["sql"],
 "additionalProperties": false
}
```

## Success Response

```
{
 "success": true,
 "tool": "query_bigquery",
 "data": {
 "rows": [
 { "user_id": 1, "email": "alice@example.com", "created_at": "2024-01-15T10:30:00" }
],
 "row_count": 1,
 "schema": [
 { "name": "user_id", "field_type": "INTEGER", "mode": "NULLABLE" },
 { "name": "email", "field_type": "STRING", "mode": "NULLABLE" },
 { "name": "created_at", "field_type": "TIMESTAMP", "mode": "NULLABLE" }
]
 },
 "meta": {
 "job_id": "project:US.bq-job-abc123",
 "bytes_processed": 4096,
 "execution_time_ms": 412.3,
 "project": "my-gcp-project"
 }
}
```

## Error Response (non-SELECT SQL)

```
{
 "success": false,
 "error_code": "VALIDATION_ERROR",
 "message": "Only SELECT statements are permitted. Detected statement type: Drop. INSERT, UPDATE, DELETE, MERGE, CREATE, DROP, ALTER, and TRUNCATE are blocked.",
 "tool": "query_bigquery",
 "details": {
 "detected_type": "Drop",
 "sql_preview": "DROP TABLE `project.dataset.table`"
 }
}
```

## Error Response (cost limit exceeded)

```
{
 "success": false,
 "error_code": "SAFETY_LIMIT_EXCEEDED",
 "message": "Query would scan 209,715,200 bytes (200.0 MB), exceeding the configured limit of 104,857,600 bytes (100.0 MB). Refine the query with a WHERE clause or partition filter to reduce scan volume.",
 "tool": "query_bigquery",
 "details": {
 "estimated_bytes": 209715200,
 "limit_bytes": 104857600,
 "estimated_mb": 200.0,
 "limit_mb": 100.0
 }
}
```

---

## 2. `list_gcs_objects`

Lists objects in a GCS bucket under a given prefix.

### Input Schema

```
{
 "type": "object",
 "properties": {
 "bucket": {
 "type": "string",
 "description": "GCS bucket name (without gs:// prefix).",
 "minLength": 1
 },
 "prefix": {
 "type": "string",
 "description": "Object name prefix (e.g. 'data/2024/'). Empty string lists all objects."
 }
 },
 "required": ["bucket", "prefix"],
 "additionalProperties": false
}
```

### Example Call

```
{ "bucket": "my-data-bucket", "prefix": "reports/2024/" }
```

### Success Response

```
{
 "success": true,
 "tool": "list_gcs_objects",
 "data": {
 "objects": [
 {
 "name": "reports/2024/q1_summary.csv",
 "size_bytes": 14823,
 "content_type": "text/csv",
 "updated": "2024-04-01T09:15:00+00:00"
 },
 {
 "name": "reports/2024/q2_summary.csv",
 "size_bytes": 16004,
 "content_type": "text/csv",
 "updated": "2024-07-01T10:00:00+00:00"
 }
],
 "object_count": 2
 },
 "meta": {
 "bucket": "my-data-bucket",
 "prefix": "reports/2024/"
 }
}
```

### Error Response (bucket not found)

```
{
 "success": false,
 "error_code": "DOWNSTREAM_ERROR",
 "message": "GCS bucket 'nonexistent-bucket' was not found or you do not have access to it.",
 "tool": "list_gcs_objects",
 "details": { "bucket": "nonexistent-bucket" }
}
```

---

## 3. read\_gcs\_object

Reads the content of a single GCS object. Files exceeding the size limit are rejected before download.

### Input Schema

```
{
 "type": "object",
 "properties": {
 "bucket": {
```

```
 "type": "string",
 "description": "GCS bucket name (without gs:// prefix).",
 "minLength": 1
 },
 "path": {
 "type": "string",
 "description": "Full object path within the bucket (e.g.
'data/report.csv').",
 "minLength": 1
 }
},
"required": ["bucket", "path"],
"additionalProperties": false
}
```

### Example Call

```
{ "bucket": "my-data-bucket", "path": "config/settings.json" }
```

### Success Response

```
{
 "success": true,
 "tool": "read_gcs_object",
 "data": {
 "content": "{\"env\": \"production\", \"max_retries\": 3}",
 "encoding": "utf-8",
 "size_bytes": 42
 },
 "meta": {
 "bucket": "my-data-bucket",
 "path": "config/settings.json",
 "content_type": "application/json",
 "updated": "2024-05-20T08:00:00+00:00"
 }
}
```

### Error Response (file too large)

```
{
 "success": false,
 "error_code": "SAFETY_LIMIT_EXCEEDED",
 "message": "File at 'dumps/full_export.parquet' is 52,428,800 bytes,
which exceeds the configured maximum of 10,485,760 bytes (10.0 MB).
Increase GCS_MAX_FILE_SIZE_BYTES in .env if you need to read larger
files.",
 "tool": "read_gcs_object",
}
```

```
"details": {
 "file_size_bytes": 52428800,
 "limit_bytes": 10485760,
 "path": "dumps/full_export.parquet"
}
```

---

#### 4. `send_slack_message`

Sends a message to a Slack channel. **STUBBED** — logs and prints without calling the real API. A token-bucket rate limiter is enforced regardless.

##### Input Schema

```
{
 "type": "object",
 "properties": {
 "channel": {
 "type": "string",
 "description": "Slack channel name or ID (e.g. '#general' or 'C01234ABCDE').",
 "minLength": 1
 },
 "message": {
 "type": "string",
 "description": "Message text to send.",
 "minLength": 1
 }
 },
 "required": ["channel", "message"],
 "additionalProperties": false
}
```

##### Example Call

```
{
 "channel": "#data-alerts",
 "message": "BigQuery job bq-job-abc123 completed. Rows processed: 50,000."
}
```

##### Success Response

```
{
 "success": true,
 "tool": "send_slack_message",
}
```



```

"data": {
 "channel": "#data-alerts",
 "message": "BigQuery job bq-job-abc123 completed. Rows processed:
50,000.",
 "status": "stubbed_sent"
},
"meta": {
 "note": "Slack API call is stubbed. No real message was delivered."
}
}

```

### Error Response (rate limit exceeded)

```

{
 "success": false,
 "error_code": "SAFETY_LIMIT_EXCEEDED",
 "message": "Rate limit exceeded for channel '#data-alerts'. Maximum 5
messages per 60 seconds. Retry in approximately 47.3 seconds.",
 "tool": "send_slack_message",
 "details": {
 "channel": "#data-alerts",
 "tokens_remaining": 0.0,
 "max_calls": 5,
 "retry_after_seconds": 47.3
 }
}

```

---

## Deliverable: `error\_format.md`

```markdown

Canonical Error Envelope – MCP Enterprise Gateway

This document is the single source of truth for the structured error envelope used by every tool in this server. Future MCP servers built at Wohlig must adopt this same envelope to maintain consistency across the firm's tool ecosystem.

Error Envelope Schema

```json

```

{
 "success": false,
 "error_code": "<string>",
 "message": "<string>",
 "tool": "<string>",

```

```
"details": { "<key>": "<value>" }
}
```

| Field      | Type    | Required | Description                                                    |
|------------|---------|----------|----------------------------------------------------------------|
| success    | boolean | ✔        | Always <b>false</b> for error envelopes.                       |
| error_code | string  | ✔        | Machine-readable error category (see below).                   |
| message    | string  | ✔        | Human-readable explanation. Actionable where possible.         |
| tool       | string  | ✔        | Name of the tool that raised this error.                       |
| details    | object  | ✔        | Extra context. May be an empty <b>{}</b> . Never <b>null</b> . |

### Success Envelope Schema

```
{
 "success": true,
 "tool": "<string>",
 "data": "<any>",
 "meta": { "<key>": "<value>" }
}
```

| Field   | Type    | Required | Description                                                    |
|---------|---------|----------|----------------------------------------------------------------|
| success | boolean | ✔        | Always <b>true</b> for success envelopes.                      |
| tool    | string  | ✔        | Name of the tool that produced this result.                    |
| data    | any     | ✔        | Tool-specific payload (rows, object list, content, etc.).      |
| meta    | object  | ✔        | Optional metadata (job_id, timing, counts). May be <b>{}</b> . |

### Error Code Reference

#### VALIDATION\_ERROR

The request was malformed or failed input validation **before** any external API was called. The server performed no external operation.

**When it occurs:**

- Required argument is missing or has the wrong type.
- SQL is not a single SELECT statement (blocked DML, multi-statement, parse failure).
- String argument is empty when a non-empty value is required.

**Example:**

```
{
 "success": false,
 "error_code": "VALIDATION_ERROR",
 "message": "Only SELECT statements are permitted. Detected statement type: Insert.",
 "tool": "query_bigquery",
 "details": {
 "detected_type": "Insert",
 "sql_preview": "INSERT INTO t (col) VALUES (1)"
 }
}
```

---

## SAFETY\_LIMIT\_EXCEEDED

The request was structurally valid but was blocked by a configured safety gate **before** the primary operation was executed.

### When it occurs:

- BigQuery dry-run estimates the query would scan more than `BQ_MAX_BYTES_SCANNED` bytes.
- GCS object metadata reports a file size greater than `GCS_MAX_FILE_SIZE_BYTES`.
- Slack rate limiter has no tokens remaining for the target channel.

### Example (BigQuery cost gate):

```
{
 "success": false,
 "error_code": "SAFETY_LIMIT_EXCEEDED",
 "message": "Query would scan 209,715,200 bytes (200.0 MB), exceeding the configured limit of 104,857,600 bytes (100.0 MB).",
 "tool": "query_bigquery",
 "details": {
 "estimated_bytes": 209715200,
 "limit_bytes": 104857600
 }
}
```

### Example (GCS size gate):

```
{
 "success": false,
 "error_code": "SAFETY_LIMIT_EXCEEDED",
 "message": "File at 'exports/dump.parquet' is 52,428,800 bytes, exceeding the maximum of 10,485,760 bytes.",
 "tool": "read_gcs_object",
 "details": {
 "file_size_bytes": 52428800,
 }
}
```

```
 "limit_bytes": 10485760,
 "path": "exports/dump.parquet"
 }
}
```

### Example (Slack rate limit):

```
{
 "success": false,
 "error_code": "SAFETY_LIMIT_EXCEEDED",
 "message": "Rate limit exceeded for channel '#alerts'. Retry in
approximately 32.5 seconds.",
 "tool": "send_slack_message",
 "details": {
 "channel": "#alerts",
 "tokens_remaining": 0.0,
 "max_calls": 5,
 "retry_after_seconds": 32.5
 }
}
```

---

## DOWNSTREAM\_ERROR

A valid request, cleared through all safety gates, failed because of an error in the external system (GCP API, network, auth). The server did not crash; it caught the exception and returned this envelope.

### When it occurs:

- `google.cloud.exceptions.GoogleCloudError` from BigQuery or GCS.
- GCS object or bucket does not exist (`NotFound`).
- Auth failure (`Forbidden`, credential not configured).
- Network timeout or transient infrastructure failure.

### Example:

```
{
 "success": false,
 "error_code": "DOWNSTREAM_ERROR",
 "message": "GCS bucket 'my-bucket' was not found or you do not have
access to it.",
 "tool": "list_gcs_objects",
 "details": {
 "bucket": "my-bucket"
 }
}
```

# Design Rationale

## Why Three Error Codes?

Three codes map cleanly to three response strategies for the LLM client:

| Code                  | Client Strategy                                                      |
|-----------------------|----------------------------------------------------------------------|
| VALIDATION_ERROR      | Reject and ask user to reformulate the request.                      |
| SAFETY_LIMIT_EXCEEDED | Reject with an explanation; may suggest user adjusts query or waits. |
| DOWNSTREAM_ERROR      | Retry after backoff or report infrastructure issue to a human.       |

A client that uses `DOWNSTREAM_ERROR` as a retry signal must never retry on `VALIDATION_ERROR` — the same call will produce the same error indefinitely.

## Why `details` is Always an Object?

Returning `"details": {}` instead of `"details": null` means consuming code can always do `error["details"].get("key")` without a null check. Consistency reduces bugs in client code.

## Extending This Schema for Future Servers

New error codes must be documented here before being introduced. Suggested additions for future MCP servers:

- `AUTH_ERROR` — for auth failures that are distinct from generic downstream failures.
- `QUOTA_EXCEEDED` — for quota-specific rejections (e.g., BigQuery slot quota).
- `TIMEOUT_ERROR` — for operations that exceeded a configured timeout.

```

6. Intern Viva & Code Review Questions (`questions.md`)

```markdown
## Project Evaluation & Code Review

### Q1: What is the Model Context Protocol (MCP) and what is the strict responsibility boundary between an MCP server and its client?
**Answer:**
MCP is an open protocol that standardises how LLM-powered applications expose tools to AI models. An MCP server is responsible for exactly one thing: exposing a catalogue of typed, schema-validated tools and executing them when called. It never reasons, never decides which tool to call, and never chains operations together autonomously. The MCP client (Claude Desktop, Cursor, a custom agent loop) holds the conversation, sends instructions to an LLM, and decides which tool to invoke based on the model's response. This boundary is non-negotiable: placing reasoning logic in the server conflates two concerns, makes the server impossible to audit tool-by-tool, and breaks the MCP contract that every tool invocation is a
```

discrete, governed transaction.

Q2: Walk through what happens when the MCP client calls ``query_bigquery`` with a valid SELECT statement that scans 50 MB.

****Answer:****

1. The client sends a JSON-RPC ``call_tool`` message with ``name: "query_bigquery"`` and ``arguments: {sql: "SELECT ..."}``. The MCP SDK deserialises this and invokes the ``call_tool`` handler in ``server.py``.
2. ``server.py`` verifies that ``arguments["sql"]`` is a string, then calls ``query_bigquery.run(sql=sql)``.
3. ``run()`` calls ``validate_select_only(sql)``. ``sqlglot.parse()`` produces an AST; the check confirms exactly one statement exists and its root node is ``exp.Select``. Returns ``None`` (no error).
4. ``run()`` creates a ``bigquery.Client()`` and calls ``client.query(sql, job_config=QueryJobConfig(dry_run=True, use_query_cache=False))``. The BigQuery API returns a job object with ``total_bytes_processed = 52_428_800`` (50 MB). This is below ``BQ_MAX_BYTES_SCANNED`` (100 MB), so no error.
5. ``run()`` calls ``client.query(sql)`` for real execution. ``job.result()`` returns rows. Rows are serialised via ``_serialize_row()`` (handling datetime/Decimal/bytes). A ``make_success()`` envelope is built with rows, schema, job_id, bytes_processed, and execution_time_ms.
6. ``server.py`` serialises the envelope to JSON and returns it as ``types.TextContent``. The MCP SDK sends it back to the client.
7. The log file (``output.log``) contains entries for invocation, dry-run completion, and success with all metadata.

Q3: Why is regex insufficient for SQL safety validation, and what specific attacks does sqlglot's AST approach prevent?

****Answer:****

Regex operates on the raw text of the query. It cannot understand SQL's structure, so it is bypassed by:

- ****Case variation****: ``DrOp TaBlE`` passes a case-insensitive check that anchors on ``^SELECT``.
- ****Comment injection****: ``/* DROP TABLE t */ SELECT 1`` – the DROP is hidden in a comment; regex anchored on word boundaries still triggers on the word "DROP" but only if you remembered to check inside comments (most implementations don't).
- ****Multi-statement batches****: ``SELECT 1; DROP TABLE t`` – a regex checking that the string starts with SELECT and does not contain DROP will miss the second statement after the semicolon if the check is ``re.match`` (anchored to start).
- ****CTE-wrapped DML****: ``WITH cte AS (DELETE FROM t RETURNING *) SELECT * FROM cte`` – this starts with WITH and ends with SELECT, passing most regex checks while executing a DELETE.

``sqlglot.parse()`` builds a full AST. The check then verifies: (a) exactly one statement node exists in the parse result (multi-statement detection), (b) the root statement node is ``exp.Select`` (type detection), and (c) a recursive walk of the AST finds no ``Insert``, ``Delete``, ``Update``, ``Merge``, ``Drop``, ``Create``, ``AlterTable``, or ``TruncateTable`` nodes anywhere in the

tree (DML-in-CTE detection). None of the above attacks can circumvent a correctly implemented AST walk.

Q4: Explain BigQuery dry-run cost estimation. What does it measure, what are its limitations, and why is `use\_query\_cache=False` set?

Answer:

A BigQuery dry-run sends the query to BigQuery's planner without executing it. The planner resolves table references, applies partition pruning, and estimates how many bytes of data would be scanned if the query ran. This estimate is returned in `job.total\_bytes\_processed` within milliseconds.

Limitations: (1) The estimate is an upper bound for partitioned and clustered tables – actual bytes scanned at execution may be lower after runtime pruning. (2) The dry-run validates syntax and table existence but not all runtime semantic errors (e.g., some function argument type mismatches). (3) Queries over streaming buffers may report slightly different estimates vs execution. (4) For cached results, the actual bytes billed is zero – but `use\_query\_cache=False` ensures we measure the true scan volume rather than getting a misleading zero estimate when the result is cached. A zero estimate would bypass the cost gate and allow an unlimited-scan query to execute unchecked on cache miss.

Q5: Why does `read\_gcs\_object` check `blob.size` before calling `blob.download\_as\_bytes()`, and what would go wrong if the order were reversed?

Answer:

`bucket.get\_blob(path)` issues an HTTP metadata request (similar to `HEAD`) that populates `blob.size`, `blob.content\_type`, `blob.updated`, and other attributes without transferring any object content. The `check\_blob\_size()` call then compares this size against `GCS\_MAX\_FILE\_SIZE\_BYTES`. Only if the check passes does the code call `blob.download\_as\_bytes()`.

If the order were reversed – download first, then check size – three problems arise: (1) Memory consumption: a 500 MB file would be fully loaded into the Python process's heap before being rejected. (2) Egress cost and latency: GCS charges for outbound bandwidth; downloading a multi-GB file just to reject it wastes money and takes seconds to minutes. (3) OOM risk: the server process could be killed by the OS if the file is large enough to exhaust available memory, causing a crash that violates the graceful-degradation policy. The metadata-first approach means rejections are free, instantaneous, and memory-safe.

Q6: Compare token bucket and fixed-window rate limiting. Why was the token bucket chosen for the Slack stub, and what problem does it solve that fixed windows cannot?

Answer:

A fixed-window limiter resets a counter to zero at the start of each time window (e.g., every 60 seconds). If the limit is 5 per 60 seconds, an

attacker can send 5 messages at 11:00:59 and 5 more at 11:01:00 – 10 messages in 1 second – because the counter reset at the minute boundary. This is the "boundary burst" problem.

A token bucket avoids this by maintaining a continuous counter of available tokens. Tokens refill at a constant rate ($\text{max_calls} / \text{period_seconds}$ per second), capped at `max_calls`. Each call consumes one token. A burst depletes the bucket and cannot be repeated until tokens refill. There is no special reset moment – the rate is enforced as a true average over any arbitrary time window.

The token bucket was chosen because: (1) it provides smooth rate limiting without boundary bursts, (2) it allows short legitimate bursts (up to `max_calls`) which is appropriate for Slack where an alert storm may legitimately produce 5 messages quickly, and (3) the implementation is straightforward in a single-process server without requiring an external store.

Q7: The error-handling policy distinguishes fail-fast from graceful degradation. Define each, state which situations each applies to in this project, and explain why mixing them would be harmful.

****Answer:****

****Fail-fast**** means detecting an invalid or unsafe condition immediately, before any external operation is attempted, and returning a structured error without proceeding. In this project it applies to: malformed input (wrong types, empty strings), SQL that is not a single SELECT (sqlglot AST check), BigQuery queries that exceed the byte-scan limit (dry-run check), GCS files that exceed the size limit (metadata check), and Slack messages that exceed the rate limit (token bucket check). The shared characteristic is that the server has enough information locally to make the rejection decision without an external call.

****Graceful degradation**** means that when an external system call fails (network error, auth failure, API quota, backend crash), the exception is caught, logged, and returned as a structured `DOWNSTREAM_ERROR` envelope. The server process does not crash. In this project it applies to: `GoogleCloudError` from BigQuery or GCS after the safety gates have passed, `NotFound` from GCS for a missing object, and any `Exception` from an unexpected external failure.

Mixing them would be harmful because: applying graceful degradation to input validation would mean invalid inputs silently reach external APIs (wasted cost, potential data modification on a misconfigured call). Applying fail-fast to downstream failures would crash the server process on a transient network blip, making the server unavailable for subsequent valid calls. The policy keeps the server both strict on inputs and resilient on infrastructure.

Q8: Describe the mocking strategy used in the pytest suite. Why must tests never make real network calls, and what specific mock patterns are

used for BigQuery and GCS?

****Answer:****

The tests use ``unittest.mock.patch`` (via ``pytest-mock``'s compatible API) to replace ``google.cloud.bigquery.Client`` and ``google.cloud.storage.Client`` with ``MagicMock`` instances at the point of import in the tool modules. This is done with ``@patch("tools.query_bigquery.bigquery.Client")``, which patches the name at the module level where it is used (not at the original definition location) – a critical distinction to get mocking right.

Real network calls must be avoided in tests because: (1) tests become non-deterministic (they depend on external state that can change), (2) tests incur real costs (BigQuery scan charges, GCS egress), (3) tests require valid credentials and network access, making them impossible to run in CI/CD without injecting secrets, and (4) tests run orders of magnitude slower over a real network than against mocks.

For BigQuery, ``mock_client.query.side_effect = [dry_run_job, exec_job]`` is used – the first call returns a mock dry-run job with a controlled ``total_bytes_processed`` value, and the second returns a mock execution job with controlled rows and schema. ``PropertyMock`` is used for ``total_bytes_processed`` because it is a property on the real ``QueryJob`` object, not a plain attribute. For GCS, ``mock_client.bucket.return_value.get_blob.return_value = blob`` sets up the metadata fetch, and ``blob.download_as_bytes.return_value = content`` sets up the download. ``blob.download_as_bytes.assert_not_called()`` verifies the pre-download size gate worked correctly.

Q9: What are the security implications of stdio transport versus SSE/HTTP transport for an MCP server, and when would you choose each?

****Answer:****

****stdio transport**:** The server runs as a subprocess spawned by the client application. Access is controlled entirely by OS process permissions – only the parent process can communicate with the server over its stdin/stdout pipes. There is no network socket, no port to firewall, no authentication token for the transport layer. Credentials (API keys, ADC) are inherited from the user's shell environment. This is highly secure for single-user, local deployments because the attack surface is limited to the local process namespace. It is the correct choice for IDE integrations (Claude Desktop, Cursor) and development environments.

****SSE/HTTP transport**:** The server listens on a TCP port and accepts connections from any HTTP client that can reach it. This introduces a network attack surface: the port must be firewalled appropriately, requests must be authenticated (e.g., bearer tokens), and TLS must be used to prevent eavesdropping and MITM attacks. SSE/HTTP is appropriate when the server is shared across multiple users or machines (team deployment), when the client cannot spawn subprocesses, or when the server needs to be deployed behind a load balancer.

For this project, stdio is correct: it is a single-developer setup, the client (Claude Desktop or Cursor) can spawn subprocesses, and the security model of inherited shell credentials is appropriate. Switching to SSE/HTTP

would require adding auth middleware, TLS configuration, and a more complex deployment, with no benefit at this stage.

Q10: If you were extending this server to production with real Slack API calls, what additional concerns would you address in `send_slack_message`, and how would you modify the rate limiter for a multi-process deployment?

Answer:

For real Slack API calls: (1) **Auth**: inject `SLACK_BOT_TOKEN` via environment variable; never hardcode it. Initialise `slack_sdk.WebClient` with that token. (2) **Error handling**: Slack's API returns rate-limit errors as HTTP 429 with a `Retry-After` header; catch `SlackApiError` with status 429 separately from generic errors and surface the `Retry-After` value in the error details. (3) **Message length**: Slack's API rejects messages over 3,000 characters in a single block; add a `VALIDATION_ERROR` check for message length. (4) **Channel validation**: validate that the channel string is a valid Slack channel ID (`C...`) or name (`#...`) format before calling the API. (5) **Idempotency**: for critical alerts, consider including a deduplication key and checking whether the same message was recently sent to avoid duplicates on retry.

For a multi-process deployment, the in-memory token bucket breaks because each process maintains independent state – five processes each with a limit of 5 calls effectively gives 25 calls per window. The fix is to move the rate limiter state to a shared store. Redis is the standard choice: use `INCR` with `EXPIRE` for a sliding window counter, or use a Lua script to implement the token bucket atomically. The `check_and_consume` function would become an async call to Redis (`await redis.eval(lua_script, channel, max_calls, rate)`) instead of operating on a local dict. Alternatively, use a distributed rate-limiting library such as `limits` (Python) with a Redis backend, which handles the atomic operations internally.